8 *Nonparametric Methods*

In the previous chapters, we discussed the parametric and semiparametric approaches where we assumed that the data is drawn from one or a mixture of probability distributions of known form. Now, we discuss the nonparametric approach that is used when no such assumption can be made about the input density and the data speaks for itself. We consider the nonparametric approaches for density estimation, classification, outlier detection, and regression and see how the time and space complexity can be checked.

8.1 Introduction

IN PARAMETRIC methods, whether for density estimation, classification, or regression, we assume a model valid over the whole input space. In regression, for example, when we assume a linear model, we assume that for any input, the output is the same linear function of the input. In classification when we assume a normal density, we assume that all examples of the class are drawn from this same density. The advantage of a parametric method is that it reduces the problem of estimating a probability density function, discriminant, or regression function to estimating the values of a small number of parameters. Its disadvantage is that this assumption does not always hold and we may

incur a large error if it does not. If we cannot make such assumptions and cannot come up with a parametric model, one possibility is to use a semiparametric mixture model as we saw in chapter 7 where the density is written as a disjunction of a small number of parametric models.

In ***nonparametric estimation***, all we assume is that *similar inputs have similar outputs*. This is a reasonable assumption: The world is smooth, and functions, whether they are densities, discriminants, or regression functions, change slowly. Similar instances mean similar things. We all love our neighbors because they are so much like us.

Therefore, our algorithm is composed of finding the similar past instances from the training set using a suitable distance measure and interpolating from them to find the right output. Different nonparametric methods differ in the way they define similarity or interpolate from the similar training instances. In a parametric model, all of the training instances affect the final global estimate, whereas in the nonparametric case, there is no single global model; local models are estimated as they are needed, affected only by the nearby training instances.

Nonparametric methods do not assume any a priori parametric form for the underlying densities; in a looser interpretation, a nonparametric model is not fixed but its complexity depends on the size of the training set, or rather, the complexity of the problem inherent in the data.

In machine learning literature, nonparametric methods are also called ***instance-based*** or ***memory-based learning*** algorithms, since what they do is store the training instances in a lookup table and interpolate from these. This implies that all of the training instances should be stored and storing all requires memory of $O(N)$. Furthermore, given an input, similar ones should be found, and finding them requires computation of $O(N)$. Such methods are also called *lazy* learning algorithms, because unlike the *eager* parametric

models, they do not compute a model when they are given the training set but postpone the computation of the model until they are given a test instance. In the case of a parametric approach, the model is quite simple and has a small number of parameters, of order $O(d)$, or $O(d^2)$, and once these parameters are calculated from the training set, we keep the model and no longer need the training set to calculate the output. N is generally much larger than d (or d^2), and this increased need for memory and computation is the disadvantage of the nonparametric methods.

We start by estimating a density function, and discuss its use in classification. We then generalize the approach to regression.

8.2 Nonparametric Density Estimation

As usual in density estimation, we assume that the sample $\mathcal{X} = \{x^t\}_{t=1}^N$ is drawn independently from some unknown probability density $p(\cdot)$. $\hat{p}(\cdot)$ is our estimator of $p(\cdot)$. We start with the univariate case where x^t are scalars and later generalize to the multidimensional case.

The nonparametric estimator for the cumulative distribution function, $F(x)$, at point x is the proportion of sample points that are less than or equal to x

$$\hat{F}(x) = \frac{\#\{x^t \le x\}}{N} \tag{8.1}$$

where $\#\{x^t \le x\}$ denotes the number of training instances whose x^t is less than or equal to x. Similarly, the nonparametric estimate for the density function, which is the derivative of the cumulative distribution, can be calculated as

$$\hat{p}(x) = \frac{1}{h}\left[\frac{\#\{x^t \le x+h\} - \#\{x^t \le x\}}{N}\right] \tag{8.2}$$

h is the length of the interval and instances x^t that fall in this interval are assumed to be "close enough." The techniques given in this chapter are variants where different heuristics are used to determine the instances that are close and their effects on the estimate.

8.2.1 Histogram Estimator

The oldest and most popular method is the ***histogram*** where the input space is divided into equal-sized intervals named *bins*. Given an origin x_0 and a bin width h, the bins are the intervals $[x_0 + mh, x_0 + (m + 1)h)$ for positive and negative integers m and the estimate is given as

$$\hat{p}(x) = \frac{\#\{x^t \text{ in the same bin as } x\}}{Nh} \tag{8.3}$$

In constructing the histogram, we have to choose both an origin and a bin width. The choice of origin affects the estimate near boundaries of bins, but it is mainly the bin width that has an effect on the estimate: With small bins, the estimate is spiky, and with larger bins, the estimate is smoother (see figure 8.1). The estimate is 0 if no instance falls in a bin and there are discontinuities at bin boundaries. Still, one advantage of the histogram is that once the bin estimates are calculated and stored, we do not need to retain the training set.

The ***naive estimator*** (Silverman 1986) frees us from setting an origin. It is defined as

$$\hat{p}(x) = \frac{\#\{x - h/2 < x^t \le x + h/2\}}{Nh} \tag{8.4}$$

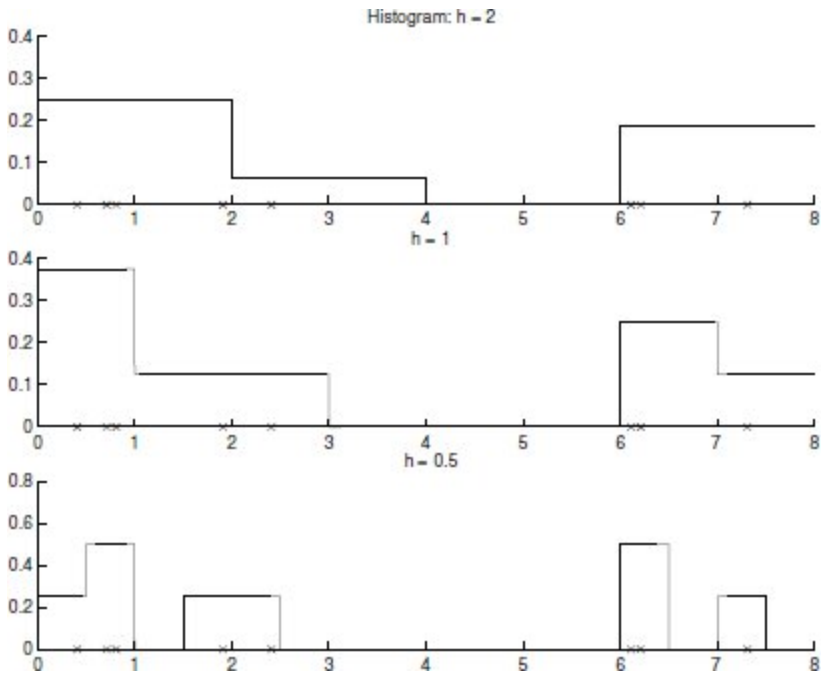


Figure 8.1 Histograms for various bin lengths. '×' denote data points.

and is equal to the histogram estimate where x is always at the center of a bin of size h (see figure 8.2). The estimator can also be written as

$$\hat{p}(x) = \frac{1}{Nh} \sum_{t=1}^{N} w\left(\frac{x - x^t}{h}\right) \tag{8.5}$$

with the *weight function* defined as

$$w(u) = \begin{cases} 1 & \text{if } |u| < 1/2 \\ 0 & \text{otherwise} \end{cases}$$

This is as if each x^t has a symmetric region of influence of size h around it and contributes 1 for an x falling in its region. Then the nonparametric estimate is just the sum of influences of x^t whose regions include x. Because this region

of influence is "hard" (0 or 1), the estimate is not a continuous function and has jumps at $x^t \pm h/2$.

8.2.2 Kernel Estimator

To get a smooth estimate, we use a smooth weight function called a ***kernel function***. The most popular is the Gaussian kernel:

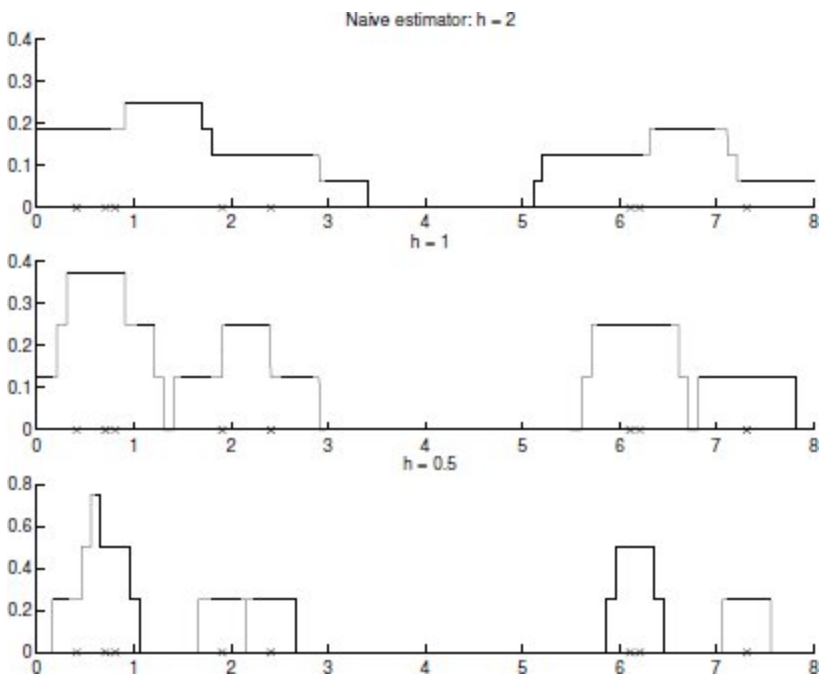


Figure 8.2 Naive estimate for various bin lengths.

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left[-\frac{u^2}{2}\right] \tag{8.6}$$

The ***kernel estimator***, also called ***Parzen windows***, is defined as

$$\hat{p}(x) = \frac{1}{Nh} \sum_{t=1}^{N} K\left(\frac{x - x^t}{h}\right) \tag{8.7}$$

The kernel function $K(\cdot)$ determines the shape of the influences and the window width h determines the width. Just like the naive estimate is the sum of "boxes," the kernel estimate is the sum of "bumps." All the x^t have an effect on the estimate at x, and this effect decreases smoothly as $|x - x^t|$ increases.

To simplify calculation, $K(\cdot)$ can be taken to be 0 if $|x - x^t| > 3h$. There exist other kernels easier to compute that can be used, as long as $K(u)$ is maximum for $u = 0$ and decreasing symmetrically as $|u|$ increases.

When h is small, each training instance has a large effect in a small region and no effect on distant points. When h is larger, there is more overlap of the kernels and we get a smoother estimate (see figure 8.3). If $K(\cdot)$ is everywhere nonnegative and integrates to 1, namely, if it is a legitimate density function, so will $\hat{p}(\cdot)$ be. Furthermore, $\hat{p}(\cdot)$ will inherit all the continuity and differentiability properties of the kernel $K(\cdot)$, so that, for example, if $K(\cdot)$ is Gaussian, then $\hat{p}(\cdot)$ will be smooth having all the derivatives.

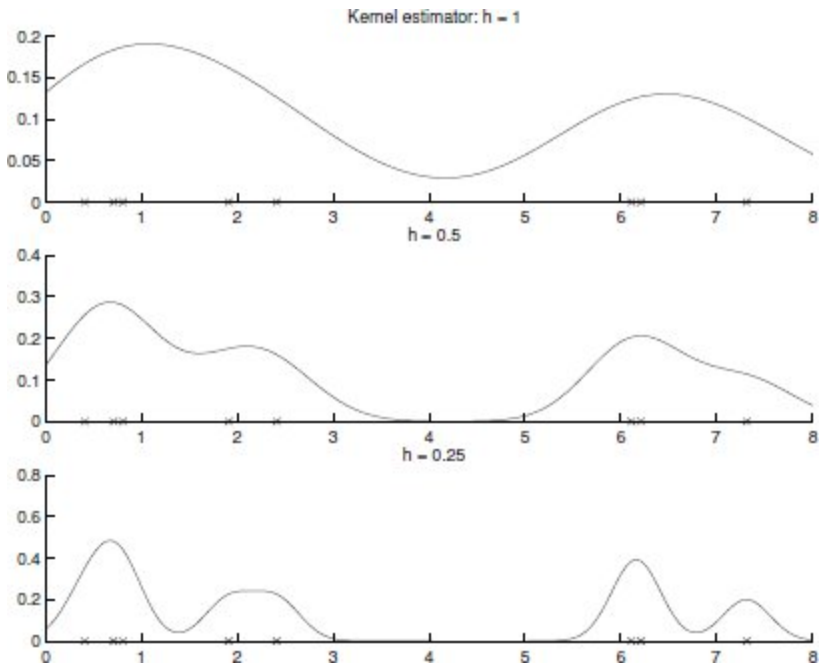


Figure 8.3 Kernel estimate for various bin lengths.

One problem is that the window width is fixed across the entire input space. Various adaptive methods have been proposed to tailor h as a function of the density around x.

8.2.3 *k*-Nearest Neighbor Estimator

The nearest neighbor class of estimators adapts the amount of smoothing to the *local* density of data. The degree of smoothing is controlled by k, the number of neighbors taken into account, which is much smaller than N, the sample size. Let us define a distance between a and b, for example, $|a - b|$, and for each x, we define

$$d_1(x) \leq d_2(x) \leq \cdots \leq d_N(x)$$

to be the distances arranged in ascending order, from x to the points in the sample: $d_1(x)$ is the distance to the nearest sample, $d_2(x)$ is the distance to the next nearest, and so on. If

x^t are the data points, then we define $d_1(x) = \min_t |x - x^t|$, and if i is the index of the closest sample, namely, $i = \arg\min_t |x - x^t|$, then $d_2(x) = \min_{j \neq i} |x - x^j|$, and so forth.

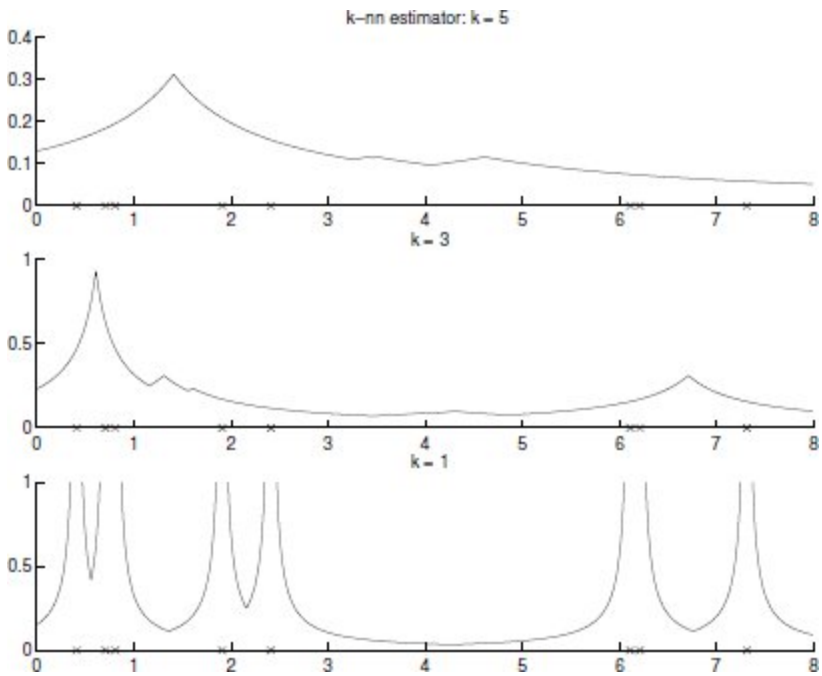


Figure 8.4 k-nearest neighbor estimate for various k values.

The ***k-nearest neighbor*** (k-nn) ***density estimate*** is

$$\hat{p}(x) = \frac{k}{2Nd_k(x)} \tag{8.8}$$

This is like a naive estimator with $h = 2d_k(x)$, the difference being that instead of fixing h and checking how many samples fall in the bin, we fix k, the number of observations to fall in the bin, and compute the bin size. Where density is high, bins are small, and where density is low, bins are larger (see figure 8.4).

The k-nn estimator is not continuous; its derivative has a discontinuity at all $\frac{1}{2}(x^{(j)} + x^{(j+k)})$ where $x^{(j)}$ are the order

statistics of the sample. The k-nn is not a probability density function since it integrates to ∞, not 1.

To get a smoother estimate, we can use a kernel function whose effect decreases with increasing distance

$$\hat{p}(x) = \frac{1}{Nd_k(x)} \sum_{t=1}^{N} K\left(\frac{x - x^t}{d_k(x)}\right) \tag{8.9}$$

This is like a kernel estimator with adaptive smoothing parameter $h = d_k(x)$. $K(\cdot)$ is typically taken to be the Gaussian kernel.

8.3 Generalization to Multivariate Data

Given a sample of d-dimensional observations $\mathcal{X} = \{\boldsymbol{x}^t\}_{t=1}^N$, the multivariate kernel density estimator is

$$\hat{p}(\boldsymbol{x}) = \frac{1}{Nh^d} \sum_{t=1}^{N} K\left(\frac{\boldsymbol{x} - \boldsymbol{x}^t}{h}\right) \tag{8.10}$$

with the requirement that

$$\int_{\Re^d} K(\boldsymbol{x}) d\boldsymbol{x} = 1$$

The obvious candidate is the multivariate Gaussian kernel:

$$K(\boldsymbol{u}) = \left(\frac{1}{\sqrt{2\pi}}\right)^d \exp\left[-\frac{\|\boldsymbol{u}\|^2}{2}\right] \tag{8.11}$$

However, care should be applied to using nonparametric estimates in high-dimensional spaces because of the ***curse of dimensionality***: Let us $\boldsymbol{x}$ is eight-dimensional, and we use a histogram with ten bins per dimension, then there are 10^8 bins, and unless we have lots of data, most of these bins will be empty and the estimates in there will be 0. In high dimensions, the concept of "close" also becomes blurry so we should be careful in choosing h.

For example, the use of the Euclidean norm in equation 8.11 implies that the kernel is scaled equally on all dimensions. If the inputs are on different scales, they should be normalized to have the same variance. Still, this does not take correlations into account, and better results are achieved when the kernel has the same form as the underlying distribution

$$K(\boldsymbol{u}) = \frac{1}{(2\pi)^{d/2}|\mathbf{S}|^{1/2}} \exp\left[-\frac{1}{2}\boldsymbol{u}^T\mathbf{S}^{-1}\boldsymbol{u}\right] \tag{8.12}$$

where **S** is the sample covariance matrix. This corresponds to using Mahalanobis distance instead of the Euclidean distance.

8.4 Nonparametric Classification

When used for classification, we use the nonparametric approach to estimate the class-conditional densities, $p(\boldsymbol{x}|C_i)$. The kernel estimator of the class-conditional density is given as

$$\hat{p}(\boldsymbol{x}|C_i) = \frac{1}{N_i h^d} \sum_{t=1}^{N} K\left(\frac{\boldsymbol{x}-\boldsymbol{x}^t}{h}\right) r_i^t \tag{8.13}$$

where r_i^t is 1 if $\boldsymbol{x}^t \in C_i$ and 0 otherwise. N_i is the number of labeled instances belonging to C_i: $N_i = \sum_t r_i^t$. The MLE of the prior density is $\hat{P}(C_i) = N_i/N$. Then, the discriminant can be written as

$$\begin{aligned} g_i(\boldsymbol{x}) &= \hat{p}(\boldsymbol{x}|C_i)\hat{P}(C_i) \\ &= \frac{1}{Nh^d} \sum_{t=1}^{N} K\left(\frac{\boldsymbol{x}-\boldsymbol{x}^t}{h}\right) r_i^t \end{aligned} \tag{8.14}$$

and $\boldsymbol{x}$ is assigned to the class for which the discriminant takes its maximum. The common factor $1/(Nh^d)$ can be ignored. So each training instance votes for its class and has no effect on other classes; the weight of vote is given by the

kernel function $K(\cdot)$, typically giving more weight to closer instances.

For the special case of k-nn estimator, we have

$$\hat{p}(\boldsymbol{x}|C_i) = \frac{k_i}{N_i V^k(\boldsymbol{x})} \tag{8.15}$$

where k_i is the number of neighbors out of the k nearest that belong to C_i and $V^k(\boldsymbol{x})$ is the volume of the d-dimensional hypersphere centered at $\boldsymbol{x}$, with radius $r = \|\boldsymbol{x} - \boldsymbol{x}_{(k)}\|$ where $\boldsymbol{x}_{(k)}$ is the k-th nearest observation to $\boldsymbol{x}$ (among all neighbors from all classes of $\boldsymbol{x}$): $V^k = r^d c_d$ with c_d as the volume of the unit sphere in d dimensions, for example, $c_1 = 2$, $c_2 = \pi$, $c_3 = 4\pi/3$, and so forth. Then

$$\hat{P}(C_i|\boldsymbol{x}) = \frac{\hat{p}(\boldsymbol{x}|C_i)\hat{P}(C_i)}{\hat{p}(\boldsymbol{x})} = \frac{k_i}{k} \tag{8.16}$$

The ***k-nn classifier*** assigns the input to the class having most examples among the k neighbors of the input. All neighbors have equal vote, and the class having the maximum number of voters among the k neighbors is chosen. Ties are broken arbitrarily or a weighted vote is taken. k is generally taken to be an odd number to minimize ties: Confusion is generally between two neighboring classes. A special case of k-nn is the ***nearest neighbor classifier*** where $k = 1$ and the input is assigned to the class of the nearest pattern. This divides the space in the form of a ***Voronoi tesselation*** (see figure 8.5).

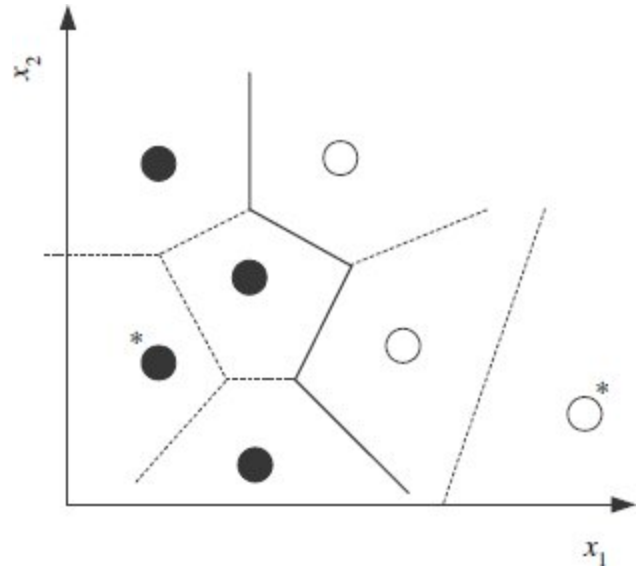


Figure 8.5 Dotted lines are the Voronoi tesselation and the straight line is the class discriminant. In condensed nearest neighbor, those instances that do not participate in defining the discriminant (marked by '*') can be removed without increasing the training error.

8.5 Condensed Nearest Neighbor

Time and space complexity of nonparametric methods are proportional to the size of the training set, and *condensing* methods have been proposed to decrease the number of stored instances without degrading performance. The idea is to select the smallest subset $\mathcal{Z}$ of $\mathcal{X}$ such that when $\mathcal{Z}$ is used in place of $\mathcal{X}$, error does not increase (Dasarathy 1991).

The best-known and earliest method is ***condensed nearest neighbor*** where 1-nn is used as the nonparametric estimator for classification (Hart 1968). 1-nn approximates the discriminant in a piecewise linear manner, and only the instances that define the discriminant need be kept; an in stance inside the class regions need not be stored as *its* nearest neighbor is of the same class and its absence does

not cause any error (on the training set) (figure 8.5). Such a subset is called a consistent subset, and we would like to find the minimal consistent subset.

```
Z ← ∅
Repeat
    For all x ∈ X (in random order)
        Find x′ ∈ Z such that ||x − x′|| = min_{x^j ∈ Z} ||x − x^j||
        If class(x)≠class(x′) add x to Z
Until Z does not change
```

Figure 8.6 Condensed nearest neighbor algorithm.

Hart proposed a greedy algorithm to find $\mathcal{Z}$ (figure 8.6). The algorithm starts with an empty $\mathcal{Z}$ and passing over the instances in $\mathcal{X}$ one by one in a random order, checks whether they can be classified correctly by 1-nn using the instances already stored in $\mathcal{Z}$. If an instance is misclassified, it is added to $\mathcal{Z}$; if it is correctly classified, $\mathcal{Z}$ is unchanged. We should pass over the training set a few times until no further instances are added. The algorithm does a local search and depending on the order in which the training instances are seen, different subsets may be found, which may have different accuracies on the validation data. Thus it does not guarantee finding the minimal consistent subset, which is known to be NP-complete (Wilfong 1992).

Condensed nearest neighbor is a greedy algorithm that aims to minimize training error and complexity, measured by the size of the stored subset. We can write an augmented error function

$$E'(\mathcal{Z}|\mathcal{X}) = E(\mathcal{X}|\mathcal{Z}) + \lambda|\mathcal{Z}| \tag{8.17}$$

where $E(\mathcal{X}|\mathcal{Z})$ is the error on $\mathcal{X}$ storing $\mathcal{Z}$. $|\mathcal{Z}|$ is the cardinality of $\mathcal{Z}$, and the second term penalizes complexity. As in any regularization scheme, λ represents the trade-off between the error and complexity such that for small λ, error becomes

more important, and as λ gets larger, complex models are penalized more. Condensed nearest neighbor is one method to minimize equation 8.17, but other algorithms to optimize it can also be devised.

8.6 Distance-Based Classification

The *k*-nearest neighbor classifier assigns an instance to the class most heavily represented among its neighbors. It is based on the idea that the more similar the instances, the more likely it is that they belong to the same class. We can use the same approach for classification as long as we have a reasonable similarity or distance measure (Chen et al. 2009).

Most classification algorithms can be recast as a distance-based classifier. For example, in section 5.5, we saw the parametric approach with Gaussian classes, and there, we talked about the *nearest mean classifier* where we choose C_i if

$$\mathcal{D}(\boldsymbol{x}, \boldsymbol{m}_i) = \min_{j=1}^{K} \mathcal{D}(\boldsymbol{x}, \boldsymbol{m}_j) \tag{8.18}$$

In the case of hyperspheric Gaussians where dimensions are independent and all are in the same scale, the distance measure is the Euclidean:

$$\mathcal{D}(\boldsymbol{x}, \boldsymbol{m}_i) = \|\boldsymbol{x} - \boldsymbol{m}_i\|$$

Otherwise it is the Mahalanobis distance:

$$\mathcal{D}(\boldsymbol{x}, \boldsymbol{m}_i) = (\boldsymbol{x} - \boldsymbol{m}_i)^T \mathbf{S}_i^{-1} (\boldsymbol{x} - \boldsymbol{m}_i)$$

where $\mathbf{S}_i$ is the covariance matrix of C_i.

In the semiparametric approach where each class is written as a mixture of Gaussians, we can say roughly speaking that we choose C_i if among all cluster centers of all classes, one that belongs to C_i is the closest:

$$\min_{l=1}^{k_i} \mathcal{D}(\boldsymbol{x}, \boldsymbol{m}_{il}) = \min_{j=1}^{K} \min_{l=1}^{k_j} \mathcal{D}(\boldsymbol{x}, \boldsymbol{m}_{jl}) \tag{8.19}$$

where k_j is the number of clusters of C_j and $\boldsymbol{m}_{jl}$ denotes the center of cluster l of C_j. Again, the distance used is the Euclidean or Mahalanobis depending on the shape of the clusters.

The nonparametric case can be even more flexible: Instead of having a distance measure per class or per cluster, we can have a different one for each neighborhood, that is, for each small region in the input space. In other words, we can define *locally adaptive distance functions* that we can then use in classification, for example, with k-nn (Hastie and Tibshirani 1996; Domeniconi, Peng, and Gunopulos 2002; Ramanan and Baker 2011).

The idea of ***distance learning*** is to parameterize $\mathcal{D}(\boldsymbol{x}, \boldsymbol{x}^t|\theta)$, learn θ from a labeled sample in a supervised manner, and then use it with k-nn (Bellet, Habrard, and Sebban 2013). The most common approach is to use the Mahalanobis distance:

$$\mathcal{D}(\boldsymbol{x}, \boldsymbol{x}^t|\mathbf{M}) = (\boldsymbol{x} - \boldsymbol{x}^t)^T \mathbf{M} (\boldsymbol{x} - \boldsymbol{x}^t) \tag{8.20}$$

where the parameter is the positive definite matrix $\mathbf{M}$. An example is the ***large margin nearest neighbor*** algorithm (Weinberger and Saul 2009) where $\mathbf{M}$ is estimated so that for all instances in the training set, the distance to a neighbor with the same label is always less than the distance to a neighbor with a different label—we discuss this algorithm in detail in section 14.13.

When the input dimensionality is high, to avoid overfitting, one approach is to add sparsity constraints on $\mathbf{M}$. The other approach is to use a low-rank approximation where we factor $\mathbf{M}$ as $\mathbf{L}^T\mathbf{L}$ and $\mathbf{L}$ is $k \times d$ with $k < d$. In this case:

$$
\begin{aligned}
\mathcal{D}(\boldsymbol{x}, \boldsymbol{x}^t | \mathbf{M}) &= (\boldsymbol{x} - \boldsymbol{x}^t)^T \mathbf{M} (\boldsymbol{x} - \boldsymbol{x}^t) = (\boldsymbol{x} - \boldsymbol{x}^t)^T \mathbf{L}^T \mathbf{L} (\boldsymbol{x} - \boldsymbol{x}^t) \\
&= (\mathbf{L}(\boldsymbol{x} - \boldsymbol{x}^t))^T (\mathbf{L}(\boldsymbol{x} - \boldsymbol{x}^t)) = (\mathbf{L}\boldsymbol{x} - \mathbf{L}\boldsymbol{x}^t)^T (\mathbf{L}\boldsymbol{x} - \mathbf{L}\boldsymbol{x}^t) \\
&= (\boldsymbol{z} - \boldsymbol{z}^t)^T (\boldsymbol{z} - \boldsymbol{z}^t) = \|\boldsymbol{z} - \boldsymbol{z}^t\|^2
\end{aligned} \tag{8.21}
$$

where $\boldsymbol{z} = \mathbf{L}\boldsymbol{x}$ is the k-dimensional projection of $\boldsymbol{x}$, and we learn $\mathbf{L}$ instead of $\mathbf{M}$. We see that the Mahalanobis distance in the original d-dimensional $\boldsymbol{x}$ space corresponds to the (squared) Euclidean distance in the new k-dimensional space. This implies the three-way relationship between distance estimation, dimensionality reduction, and feature extraction: The ideal distance measure is defined as the Euclidean distance in a new space whose (fewest) dimensions are extracted from the original inputs in the best possible way. This is demonstrated in figure 8.7.

With discrete data, ***Hamming distance*** that counts the number of nonmatching attributes can be used:

$$
HD(\boldsymbol{x}, \boldsymbol{x}^t) = \sum_{j=1}^{d} 1(x_j \neq x_j^t) \tag{8.22}
$$

where

$$
1(a) = \begin{cases} 1 & \text{if } a \text{ is true} \\ 0 & \text{otherwise} \end{cases}
$$

This framework can be used with application-dependent similarity or distance measures as well. We may have specialized similarity/distance scores for matching image parts in vision, sequence alignment scores in bioinformatics, and document similarity measures in natural language processing; these can all be used without explicitly needing to represent those entities as vectors and using a general-purpose distance such as the Euclidean distance. In chapter 14, we will talk about kernel functions that have a similar role.

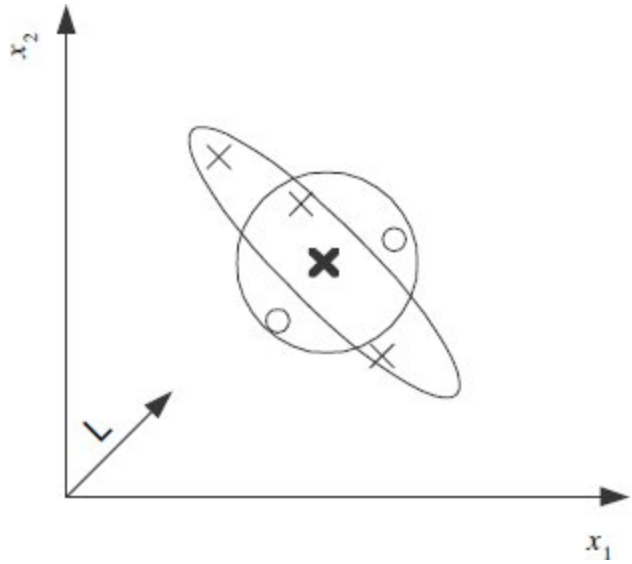


Figure 8.7 The use of Mahalanobis vs. Euclidean distance in *k*-nearest neighbor classification. There are two classes indicated by '∘' and '×'. The bold '×' is the test instance and $k = 3$. Points that are of equal Euclidean distance define a circle that here leads to misclassification. We see that there is a certain correlation structure that can be captured by the Mahalanobis distance; it defines an ellipse and leads to correct classification. We also see that if the data is projected on the direction showed by *L*, we can do correct classification in that reduced one-dimensional space.

As long as we have a similarity score function between two instances $S(\boldsymbol{x}, \boldsymbol{x}^t)$, we can define a *similarity-based representation* $\boldsymbol{x}'$ of instance $\boldsymbol{x}$ as the N-dimensional vector of scores with all the training instances, $\boldsymbol{x}^t$, $t = 1, \ldots, N$:

$$\boldsymbol{x}' = [s(\boldsymbol{x}, \boldsymbol{x}^1), s(\boldsymbol{x}, \boldsymbol{x}^2), \ldots, s(\boldsymbol{x}, \boldsymbol{x}^N)]^T$$

This can then be used as a vector to be handled by any learner (Pekalska and Duin 2002); in the context of kernel machines, we will call this the *empirical kernel map* (section 14.7).

8.7 Outlier Detection

An *outlier*, ***novelty***, or ***anomaly*** is an instance that is very much different from other instances in the sample. An outlier may indicate an abnormal behavior of the system; for example, in a dataset of credit card transactions, it may indicate fraud; in an image, outliers may indicate anomalies, for example, tumors; in a dataset of network traffic, outliers may be intrusion attempts; in a health-care scenario, an outlier indicates a significant deviation from patient's normal behavior. Outliers may also be recording errors—for example, due to faulty sensors—that should be detected and discarded to get reliable statistics.

Outlier detection is not generally cast as a supervised, two-class classification problem of separating typical instances and outliers, because generally there are very few instances that can be labeled as outliers and they do not fit a consistent pattern that can be easily captured by a two-class classifier. Instead, it is the typical instances that are modeled; this is sometimes called ***one-class classification***. Once we model the typical instances, any instance that does not fit the model (and this may occur in many different ways) is an anomaly. Another problem that generally occurs is that the data used to train the outlier detector is unlabeled and may contain outliers mixed with typical instances.

Outlier detection basically implies spotting what does not normally happen; that is, it is density estimation followed by checking for instances with too small probability under the estimated density. As usual, the fitted model can be parametric, semiparametric, or nonparametric. In the parametric case (section 5.4), for example, we can fit a Gaussian to the whole data and any instance having a low probability, or equally, with high Mahalanobis distance to the mean, is a candidate for being an outlier. In the semiparametric case (section 7.2), we fit, for example, a

mixture of Gaussians and check whether an instance has small probability; this would be an instance that is far from its nearest cluster center or one that forms a cluster by itself.

Still when the data that is used for fitting the model itself includes outliers, it makes more sense to use a nonparametric density estimator, because the more parametric a model is, the less robust it will be to the presence of outliers—for example, a single outlier may seriously corrupt the estimated mean and covariance of a Gaussian.

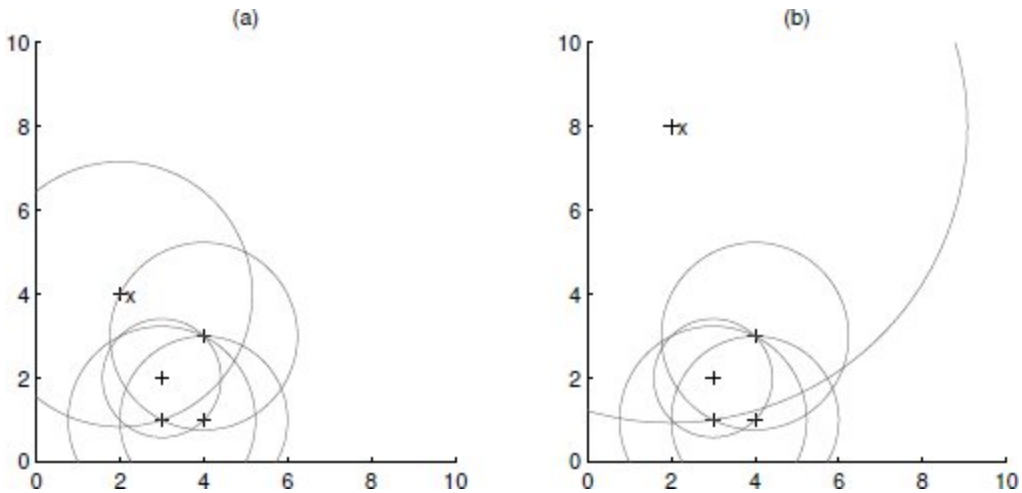


Figure 8.8 Training instances are shown by '+', '×' is the query, and the radius of the circle centered at an instance is equal to the distance to the third nearest neighbor. (a) LOF of '×' is close to 1 and it is not an outlier. (b) LOF of '×' is much larger than 1 and it is likely to be an outlier.

In nonparametric density estimation, as we discussed in the preceding sections, the estimated probability is high where there are many training instances nearby and the probability decreases as the neighborhood becomes more sparse. One example is the ***local outlier factor*** that compares the denseness of the neighborhood of an instance with the average denseness of the neighborhoods of its neighbors

(Breunig et al. 2000). Let us define $d_k(\boldsymbol{x})$ as the distance between instance $\boldsymbol{x}$ and its k-th nearest neighbor. Let us define $\mathcal{N}(\boldsymbol{x})$ as the set of training instances that are in the neighborhood of $\boldsymbol{x}$, for example, its k nearest neighbors. Consider $d_k(\boldsymbol{s})$ for $\boldsymbol{s} \in \mathcal{N}(\boldsymbol{x})$. We compare $d_k(\boldsymbol{x})$ with the average of $d_k(\boldsymbol{s})$ for such $\boldsymbol{s}$:

$$\text{LOF}(\boldsymbol{x}) = \frac{d_k(\boldsymbol{x})}{\sum_{\boldsymbol{s} \in \mathcal{N}(\boldsymbol{x})} d_k(\boldsymbol{s}) / |\mathcal{N}(\boldsymbol{x})|} \tag{8.23}$$

If LOF($\boldsymbol{x}$) is close to 1, $\boldsymbol{x}$ is not an outlier; as it gets larger, the probability that it is an outlier increases (see figure 8.8).

8.8 Nonparametric Regression: Smoothing Models

In regression, given the training set $\mathcal{X} = \{x^t, r^t\}$ where $r^t \in \Re$, we assume

$$r^t = g(x^t) + \epsilon$$

In parametric regression, we assume a polynomial of a certain order and compute its coefficients that minimize the sum of squared error on the training set. Nonparametric regression is used when no such polynomial can be assumed; we only assume that close x have close $g(x)$ values. As in nonparametric density estimation, given x, our approach is to find the neighborhood of x and average the r values in the neighborhood to calculate $\hat{g}(x)$. The nonparametric regression estimator is also called a ***smoother*** and the estimate is called a *smooth* (Hardle 1990). There are various methods for defining the neighborhood and averaging in the neighborhood, similar to methods in density estimation. We discuss the methods for the univariate x; they can be

generalized to the multivariate case in a straightforward manner using multivariate kernels, as in density estimation.

8.8.1 Running Mean Smoother

If we define an origin and a bin width and average the r values in the bin as in the histogram, we get a ***regressogram*** (see figure 8.9)

$$\hat{g}(x) = \frac{\sum_{t=1}^{N} b(x, x^t) r^t}{\sum_{t=1}^{N} b(x, x^t)} \tag{8.24}$$

where

$$b(x, x^t) = \begin{cases} 1 & \text{if } x^t \text{ is the same bin with } x \\ 0 & \text{otherwise} \end{cases}$$

Having discontinuities at bin boundaries is disturbing as is the need to fix an origin. As in the naive estimator, in the ***running mean smoother***, we define a bin symmetric around x and average in there (figure 8.10).

$$\hat{g}(x) = \frac{\sum_{t=1}^{N} w\left(\frac{x-x^t}{h}\right) r^t}{\sum_{t=1}^{N} w\left(\frac{x-x^t}{h}\right)} \tag{8.25}$$

where

$$w(u) = \begin{cases} 1 & \text{if } |u| < 1 \\ 0 & \text{otherwise} \end{cases}$$

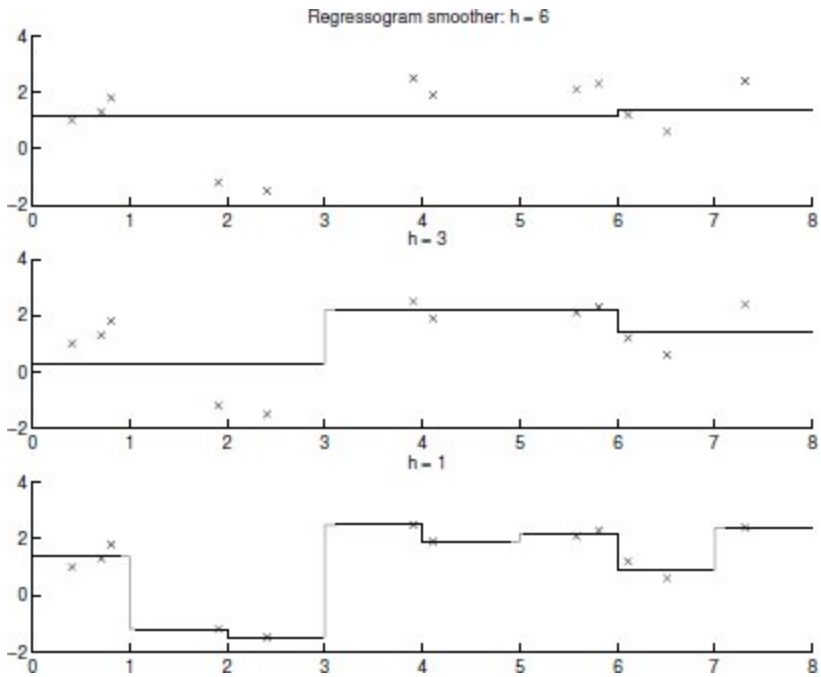


Figure 8.9 Regressograms for various bin lengths. '×' denote data points.

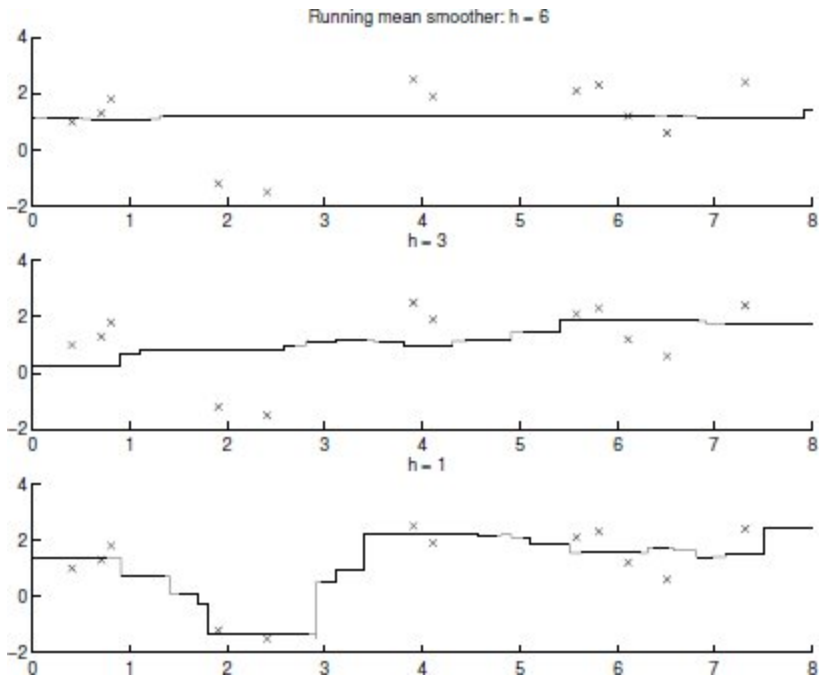


Figure 8.10 Running mean smooth for various bin lengths.

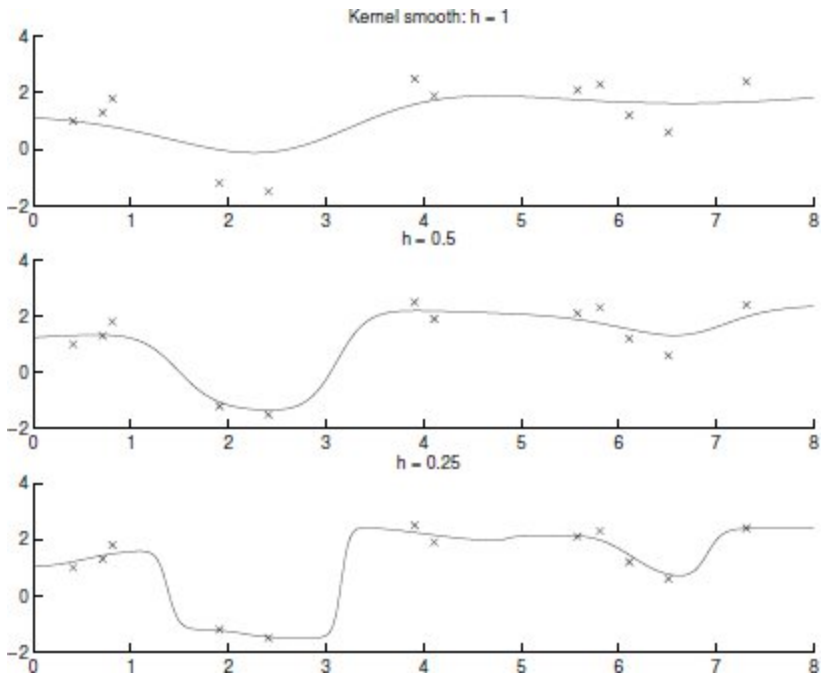


Figure 8.11 Kernel smooth for various bin lengths.

This method is especially popular with evenly spaced data, such as time series. In applications where there is noise, we can use the median of the r^t in the bin instead of their mean.

8.8.2 Kernel Smoother

As in the kernel estimator, we can use a kernel giving less weight to further points, and we get the ***kernel smoother*** (see figure 8.11):

$$\hat{g}(x) = \frac{\sum_t K\left(\frac{x-x^t}{h}\right) r^t}{\sum_t K\left(\frac{x-x^t}{h}\right)} \tag{8.26}$$

Typically a Gaussian kernel $K(\cdot)$ is used. Instead of fixing h, we can fix k, the number of neighbors, adapting the estimate to the density around x, and get the ***k-nn smoother***.

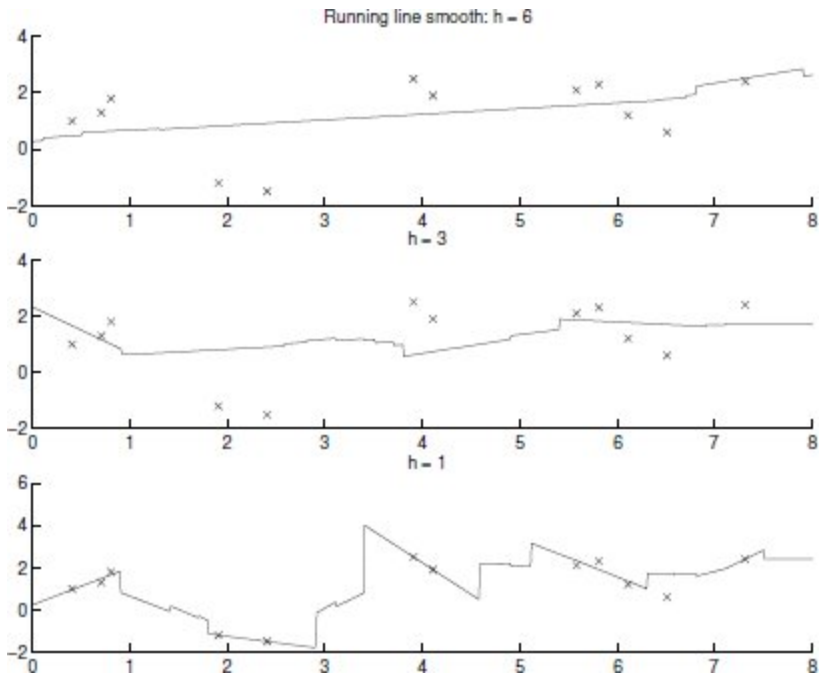


Figure 8.12 Running line smooth for various bin lengths.

8.8.3 Running Line Smoother

Instead of taking an average and giving a constant fit at a point, we can take into account one more term in the Taylor expansion and calculate a linear fit. In the ***running line smoother***, we can use the data points in the neighborhood, as defined by h or k, and fit a local regression line (see figure 8.12).

In the ***locally weighted running line smoother***, known as *loess*, instead of a hard definition of neighborhoods, we use kernel weighting such that distant points have less effect on error.

8.9 How to Choose the Smoothing Parameter

In nonparametric methods, for density estimation or regression, the critical parameter is the smoothing parameter

9 *Decision Trees*

A decision tree is a hierarchical data structure implementing the divide-and-conquer strategy. It is an efficient nonparametric method, which can be used for both classification and regression. We discuss learning algorithms that build the tree from a given labeled training sample, as well as how the tree can be converted to a set of simple rules that are easy to understand. Another possibility is to learn a rule base directly.

9.1 Introduction

IN PARAMETRIC estimation, we define a model over the whole input space and learn its parameters from all of the training data. Then we use the same model and the same parameter set for any test input. In nonparametric estimation, we divide the input space into local regions, defined by a distance measure like the Euclidean norm, and for each input, the corresponding local model computed from the training data in that region is used. In the instance-based models we discussed in chapter 8, given an input, identifying the local data defining the local model is costly; it requires calculating the distances from the given input to all of the training instances, which is $O(N)$.

A ***decision tree*** is a hierarchical model for supervised learning whereby the local region is identified in a sequence

of recursive splits in a smaller number of steps. A decision tree is composed of internal decision nodes and terminal leaves (see figure 9.1). Each ***decision node*** m implements a test function $f_m(\boldsymbol{x})$ with discrete outcomes labeling the branches. Given an input, at each node, a test is applied and one of the branches is taken depending on the outcome. This process starts at the root and is repeated recursively until a ***leaf node*** is hit, at which point the value written in the leaf constitutes the output.

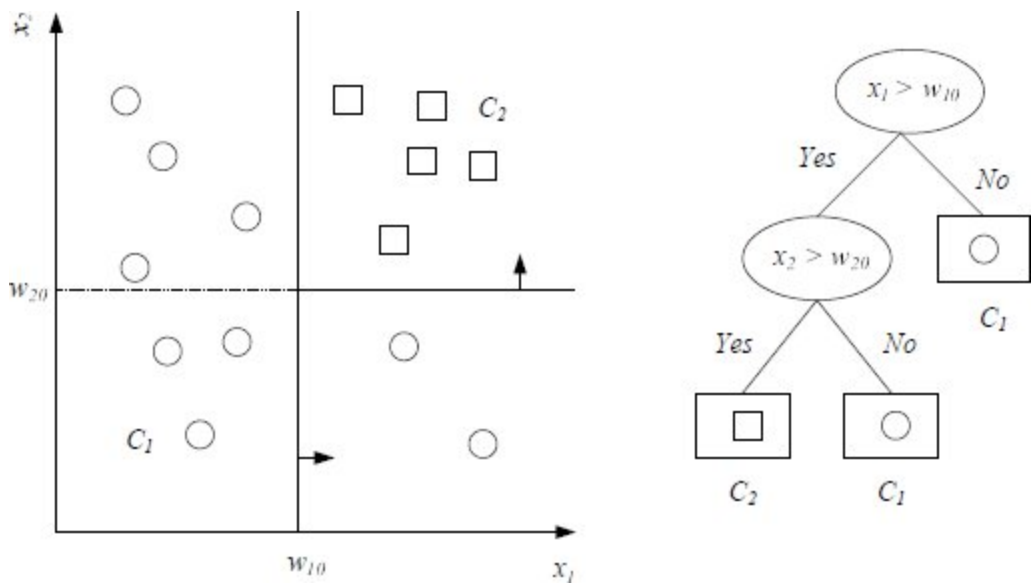


Figure 9.1 Example of a dataset and the corresponding decision tree. Oval nodes are the decision nodes and rectangles are leaf nodes. The univariate decision node splits along one axis, and successive splits are orthogonal to each other. After the first split, $\{\boldsymbol{x}|x_1 < w_{10}\}$ is pure and is not split further.

A decision tree is also a nonparametric model in the sense that we do not assume any parametric form for the class densities and the tree structure is not fixed a priori but, as we will see shortly, depending on the complexity of the

problem underlying the data, the tree grows during learning by adding new branches and leaves.

Each $f_m(\boldsymbol{x})$ defines a discriminant in the d-dimensional input space dividing it into smaller regions that are further subdivided as we take a path from the root down. $f_m(\cdot)$ is a simple function and when written as a tree, a complex function is broken down into a series of simple decisions. Different decision tree methods assume different models for $f_m(\cdot)$, and the model class defines the shape of the discriminant and the shape of regions. Each leaf node has an output label, which in the case of classification is the class code and in regression is a numeric value. A leaf node defines a localized region in the input space where instances falling in this region have the same labels (in classification), or very similar numeric outputs (in regression). The boundaries of the regions are defined by the discriminants that are coded in the internal nodes on the path from the root to the leaf node.

The hierarchical placement of decisions allows a fast localization of the region covering an input. For example, if the decisions are binary, then in the best case, each decision eliminates half of the cases. If there are b regions, then in the best case, the correct region can be found in $\log_2 b$ decisions. Another advantage of the decision tree is interpretability. As we will see shortly, the tree can be converted to a set of *IF-THEN rules* that are easily understandable. For this reason, decision trees are very popular and sometimes preferred over more accurate but less interpretable methods.

We start with univariate trees where the test in a decision node uses only one input variable and we see how such trees can be constructed for classification and regression. We later generalize this to multivariate trees where all inputs can be used in an internal node.

9.2 Univariate Trees

In a ***univariate tree***, in each internal node, the test uses only one of the input dimensions. If the used input dimension, x_j, is discrete, taking one of n possible values, the decision node checks the value of x_j and takes the corresponding branch, implementing an n-way split. For example, if an attribute is color ∈ {red, blue, green}, then a node on that attribute has three branches, each one corresponding to one of the three possible values of the attribute.

A decision node has discrete branches and a numeric input should be discretized. If x_j is numeric (ordered), the test is a comparison

$$f_m(\boldsymbol{x}) : x_j > w_{m0} \tag{9.1}$$

where w_{m0} is a suitably chosen threshold value. The decision node divides the input space into two: $L_m = \{\boldsymbol{x} | x_j > w_{m0}\}$ and $R_m = \{\boldsymbol{x} | x_j \leq w_{m0}\}$; this is called a ***binary split***. Successive decision nodes on a path from the root to a leaf further divide these into two using other attributes and generating splits orthogonal to each other. The leaf nodes define hyperrectangles in the input space (see figure 9.1).

Tree induction is the construction of the tree given a training sample. For a given training set, there exists many trees that code it with no error, and, for simplicity, we are interested in finding the smallest among them, where tree size is measured as the number of nodes in the tree and the complexity of the decision nodes. Finding the smallest tree is NP-complete (Quinlan 1986), and we are forced to use local search procedures based on heuristics that give reasonable trees in reasonable time.

Tree learning algorithms are greedy and, at each step, starting at the root with the complete training data, we look for the best split. This splits the training data into two or n, depending on whether the chosen attribute is numeric or discrete. We then continue splitting recursively with the

corresponding subset until we do not need to split anymore, at which point a leaf node is created and labeled.

9.2.1 Classification Trees

In the case of a decision tree for classification, namely, a ***classification tree***, the goodness of a split is quantified by an ***impurity measure***. A split is pure if after the split, for all branches, all the instances choosing a branch belong to the same class. Let us say for node m, N_m is the number of training instances reaching node m. For the root node, it is N. N_m^i of N_m belong to class C_i, with $\sum_i N_m^i = N_m$. Given that an instance reaches node m, the estimate for the probability of class C_i is

$$\hat{P}(C_i|\mathbf{x}, m) \equiv p_m^i = \frac{N_m^i}{N_m} \tag{9.2}$$

Node m is pure if p_m^i for all i are either 0 or 1. It is 0 when none of the instances reaching node m are of class C_i, and it is 1 if all such instances are of C_i. If the split is pure, we do not need to split any further and can add a leaf node labeled with the class for which p_m^i is 1. One possible function to measure impurity is ***entropy*** (Quinlan 1986) (see figure 9.2):

$$\mathcal{I}_m = -\sum_{i=1}^{K} p_m^i \log_2 p_m^i \tag{9.3}$$

where $0 \log 0 \equiv 0$. Entropy in information theory specifies the minimum number of bits needed to encode the class code of an instance. In a two-class problem, if $p^1 = 1$ and $p^2 = 0$, all examples are of C^1, and we do not need to send anything, and the entropy is 0. If $p^1 = p^2 = 0.5$, we need to send a bit to signal one of the two cases, and the entropy is 1. In between these two extremes, we can devise codes and use less than a

bit per message by having shorter codes for the more likely class and longer codes for the less likely. When there are $K > 2$ classes, the same discussion holds and the largest entropy is $\log_2 K$ when $p^i = 1/K$.

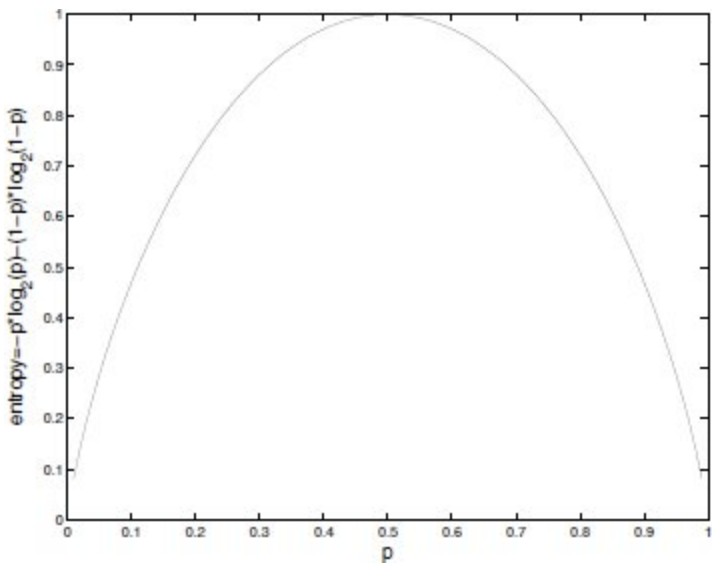


Figure 9.2 Entropy function for a two-class problem.

But entropy is not the only possible measure. For a two-class problem where $p^1 \equiv p$ and $p^2 = 1 - p$, $\phi(p, 1 - p)$ is a nonnegative function measuring the impurity of a split if it satisfies the following properties (Devroye, Györfi, and Lugosi 1996):

- $\phi(1/2, 1/2) \geq \phi(p, 1 - p)$, for any $p \in [0, 1]$.
- $\phi(0, 1) = \phi(1, 0) = 0$.
- $\phi(p, 1 - p)$ is increasing in p on $[0, 1/2]$ and decreasing in p on $[1/2, 1]$.

Examples are

1. Entropy

$$\phi(p, 1 - p) = -p \log_2 p - (1 - p) \log_2 (1 - p) \tag{9.4}$$

Equation 9.3 is the generalization to $K > 2$ classes.

2. ***Gini index*** (Breiman et al. 1984)

$$\phi(p, 1-p) = 2p(1-p) \tag{9.5}$$

3. Misclassification error

$$\phi(p, 1-p) = 1 - \max(p, 1-p) \tag{9.6}$$

These can be generalized to $K > 2$ classes, and the misclassification error can be generalized to minimum risk given a loss function (exercise 1). Research has shown that there is not a significant difference between these three measures.

If node m is not pure, then the instances should be split to decrease impurity, and there are multiple possible attributes on which we can split. For a numeric attribute, multiple split positions are possible. Among all, we look for the split that minimizes impurity after the split because we want to generate the smallest tree. If the subsets after the split are closer to pure, fewer splits (if any) will be needed afterward. Of course this is locally optimal, and we have no guarantee of finding the smallest decision tree.

Let us say at node m, N_{mj} of N_m take branch j; these are $\mathbf{x}^t$ for which the test $f_m(\mathbf{x}^t)$ returns outcome j. For a discrete attribute with n values, there are n outcomes, and for a numeric attribute, there are two outcomes ($n = 2$), in either case satisfying $\sum_{j=1}^{n} N_{mj} = N_m$. N_{mj}^i of N_{mj} belong to class C_i: $\sum_{i=1}^{K} N_{mj}^i = N_{mj}$. Similarly, $\sum_{j=1}^{n} N_{mj}^i = N_m^i$.

Then given that at node m, the test returns outcome j, the estimate for the probability of class C_i is

$$\hat{P}(C_i|\mathbf{x}, m, j) \equiv p_{mj}^i = \frac{N_{mj}^i}{N_{mj}} \tag{9.7}$$

and the total impurity after the split is given as

$$\mathcal{I}'_m = -\sum_{j=1}^{n} \frac{N_{mj}}{N_m} \sum_{i=1}^{K} p^i_{mj} \log_2 p^i_{mj} \tag{9.8}$$

In the case of a numeric attribute, to be able to calculate p^i_{mj} using equation 9.1, we also need to know w_{m0} for that node. There are $N_m - 1$ possible w_{m0} between N_m data points: We do not need to test for all (possibly infinite) points; it is enough to test, for example, at halfway between points. Note also that the best split is always between adjacent points belonging to different classes. So we try them, and the best in terms of purity is taken for the purity of the attribute. In the case of a discrete attribute, no such iteration is necessary.

```
GenerateTree(X)
    If NodeEntropy(X)< θ_I /* equation 9.3 */
      Create leaf labelled by majority class in X
      Return
    i ← SplitAttribute(X)
    For each branch of x_i
      Find X_i falling in branch
      GenerateTree(X_i)

SplitAttribute(X)
    MinEnt← MAX
    For all attributes i = 1,...,d
        If x_i is discrete with n values
          Split X into X_1,...,X_n by x_i
          e ← SplitEntropy(X_1,...,X_n) /* equation 9.8 */
          If e<MinEnt MinEnt ← e; bestf ← i
        Else /* x_i is numeric */
          For all possible splits
              Split X into X_1, X_2 on x_i
              e←SplitEntropy(X_1, X_2)
              If e<MinEnt MinEnt ← e; bestf ← i
    Return bestf
```

Figure 9.3 Classification tree construction.

So for all attributes, discrete and numeric, and for a numeric attribute for all split positions, we calculate the impurity and choose the one that has the minimum entropy, for example, as measured by equation 9.8. Then tree construction continues recursively and in parallel for all the branches that are not pure, until all are pure. This is the basis of the ***classification and regression tree*** (CART) algorithm (Breiman et al. 1984), ***ID3*** algorithm (Quinlan 1986), and its extension ***C4.5*** (Quinlan 1993). The pseudocode of the algorithm is given in figure 9.3.

It can also be said that at each step during tree construction, we choose the split that causes the largest decrease in impurity, which is the difference between the impurity of data reaching node m (equation 9.3) and the total entropy of data reaching its branches after the split (equation 9.8).

One problem is that such splitting favors attributes with many values. When there are many values, there are many branches, and the impurity can be much less. For example, if we take training index t as an attribute, the impurity measure will choose that because then the impurity of each branch is 0, although it is not a reasonable feature. Nodes with many branches are complex and go against our idea of splitting class discriminants into simple decisions. Methods have been proposed to penalize such attributes and to balance the impurity drop and the branching factor.

When there is noise, growing the tree until it is purest, we may grow a very large tree and it overfits; for example, consider the case of a mislabeled instance amid a group of correctly labeled instances. To alleviate such overfitting, tree construction ends when nodes become pure enough, namely, a subset of data is not split further if $\mathcal{I} < \theta_{\mathcal{I}}$. This implies that we do not require that p^i_{mj} be exactly 0 or 1 but close enough, with a threshold θ_p. In such a case, a leaf node is created and is labeled with the class having the highest p^i_{mj}.

θ_I (or θ_p) is the complexity parameter, like h or k of nonparametric estimation. When they are small, the variance is high and the tree grows large to reflect the training set accurately, and when they are large, variance is lower and a smaller tree roughly represents the training set and may have large bias. The ideal value depends on the cost of misclassification, as well as the costs of memory and computation.

It is generally advised that in a leaf, one stores the posterior probabilities of classes, instead of labeling the leaf with the class having the highest posterior. These probabilities may be required in later steps, for example, in calculating risks. Note that we do not need to store the instances reaching the node or the exact counts; just ratios suffice.

9.2.2 Regression Trees

A ***regression tree*** is constructed in almost the same manner as a classification tree, except that the impurity measure that is appropriate for classification is replaced by a measure appropriate for regression. Let us say for node m, $\mathcal{X}_m$ is the subset of $\mathcal{X}$ reaching node m; namely, it is the set of all $\boldsymbol{x} \in \mathcal{X}$ satisfying all the conditions in the decision nodes on the path from the root until node m. We define

$$b_m(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \boldsymbol{x} \in \mathcal{X}_m\text{: } \boldsymbol{x} \text{ reaches node } m \\ 0 & \text{otherwise} \end{cases} \tag{9.9}$$

In regression, the goodness of a split is measured by the mean square error from the estimated value. Let us say g_m is the estimated value in node m.

$$E_m = \frac{1}{N_m} \sum_t (r^t - g_m)^2 b_m(\boldsymbol{x}^t) \tag{9.10}$$

where $N_m = |\mathcal{X}_m| = \sum_t b_m(\boldsymbol{x}^t)$.

In a node, we use the mean (median if there is too much noise) of the required outputs of instances reaching the node

$$g_m = \frac{\sum_t b_m(\mathbf{x}^t) r^t}{\sum_t b_m(\mathbf{x}^t)} \tag{9.11}$$

Then equation 9.10 corresponds to the variance at m. If at a node, the error is acceptable, that is, $E_m < \theta_r$, then a leaf node is created and it stores the g_m value. Just like the regressogram of chapter 8, this creates a piecewise constant approximation with discontinuities at leaf boundaries.

If the error is not acceptable, data reaching node m is split further such that the sum of the errors in the branches is minimum. As in classification, at each node, we look for the attribute (and split threshold for a numeric attribute) that minimizes the error, and then we continue recursively.

Let us define $\mathcal{X}_{mj}$ as the subset of $\mathcal{X}_m$ taking branch j: $\cup_{j=1}^{n} \mathcal{X}_{mj} = \mathcal{X}_m$. We define

$$b_{mj}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} \in \mathcal{X}_{mj}\text{: } \mathbf{x} \text{ reaches node } m \text{ and takes branch } j \\ 0 & \text{otherwise} \end{cases} \tag{9.12}$$

g_{mj} is the estimated value in branch j of node m.

$$g_{mj} = \frac{\sum_t b_{mj}(\mathbf{x}^t) r^t}{\sum_t b_{mj}(\mathbf{x}^t)} \tag{9.13}$$

and the error after the split is

$$E'_m = \frac{1}{N_m} \sum_j \sum_t (r^t - g_{mj})^2 b_{mj}(\mathbf{x}^t) \tag{9.14}$$

The drop in error for any split is given as the difference between equation 9.10 and equation 9.14. We look for the split such that this drop is maximum or, equivalently, where equation 9.14 takes its minimum. The code given in figure 9.3 can be adapted to training a regression tree by replacing

entropy calculations with mean square error and class labels with averages.

Mean square error is one possible error function; another is worst possible error

$$E_m = \max_j \max_t |r^t - g_{mj}| b_{mj}(\mathbf{x}^t) \tag{9.15}$$

and using this, we can guarantee that the error for any instance is never larger than a given threshold.

The acceptable error threshold is the complexity parameter; when it is small, we generate large trees and risk overfitting; when it is large, we underfit and smooth too much (see figures 9.4 and 9.5).

Similar to going from running mean to running line in nonparametric regression, instead of taking an average at a leaf that implements a constant fit, we can also do a linear regression fit over the instances choosing the leaf:

$$g_m(\mathbf{x}) = \mathbf{w}_m^T \mathbf{x} + w_{m0} \tag{9.16}$$

This makes the estimate in a leaf dependent on $\mathbf{x}$ and generates smaller trees, but there is the expense of extra computation at a leaf node.

9.3 Pruning

Frequently, a node is not split further if the number of training instances reaching a node is smaller than a certain percentage of the training set—for example, 5 percent—regardless of the impurity or error. The idea is that any decision based on too few instances causes variance and thus generalization error. Stopping tree construction early on before it is full is called ***prepruning*** the tree.

Another possibility to get simpler trees is ***postpruning***, which in practice works better than prepruning. We saw before that tree growing is greedy and at each step, we make

a decision, namely, generate a decision node, and continue further on, never backtracking and trying out an alternative. The only exception is postpruning where we try to find and prune unnecessary subtrees.

In postpruning, we grow the tree full until all leaves are pure and we have no training error. We then find subtrees that cause overfitting and we prune them. From the initial labeled set, we set aside a ***pruning set***, unused during training. For each subtree, we replace it with a leaf node labeled with the training instances covered by the subtree (appropriately for classification or regression). If the leaf node does not perform worse than the subtree on the pruning set, we prune the subtree and keep the leaf node because the additional complexity of the subtree is not justified; otherwise, we keep the subtree.

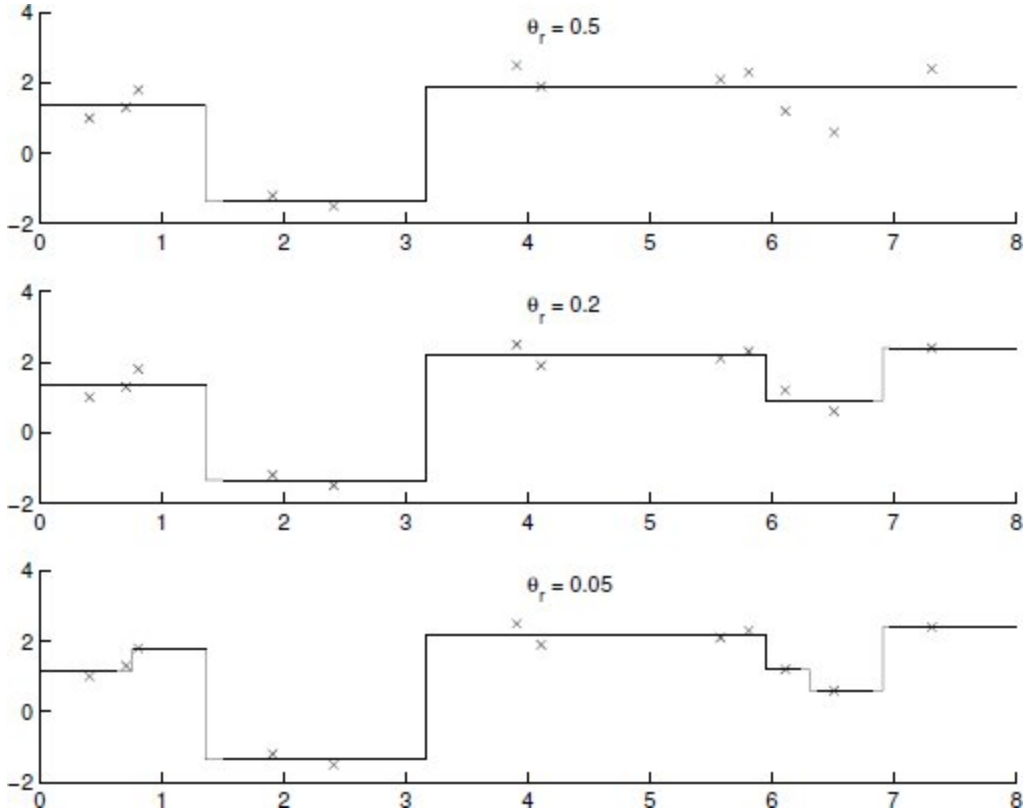
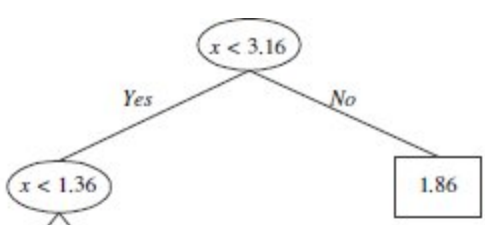


Figure 9.4 Regression tree smooths for various values of θ_r. The corresponding trees are given in figure 9.5.

For example, in the third tree of figure 9.5, there is a subtree starting with condition $x < 6.31$. This subtree can be replaced by a leaf node of $y = 0.9$ (as in the second tree) if the error on the pruning set does not increase during the substitution. Note that the pruning set should not be confused with (and is distinct from) the validation set.

Comparing prepruning and postpruning, we can say that prepruning is faster but postpruning generally leads to more accurate trees.



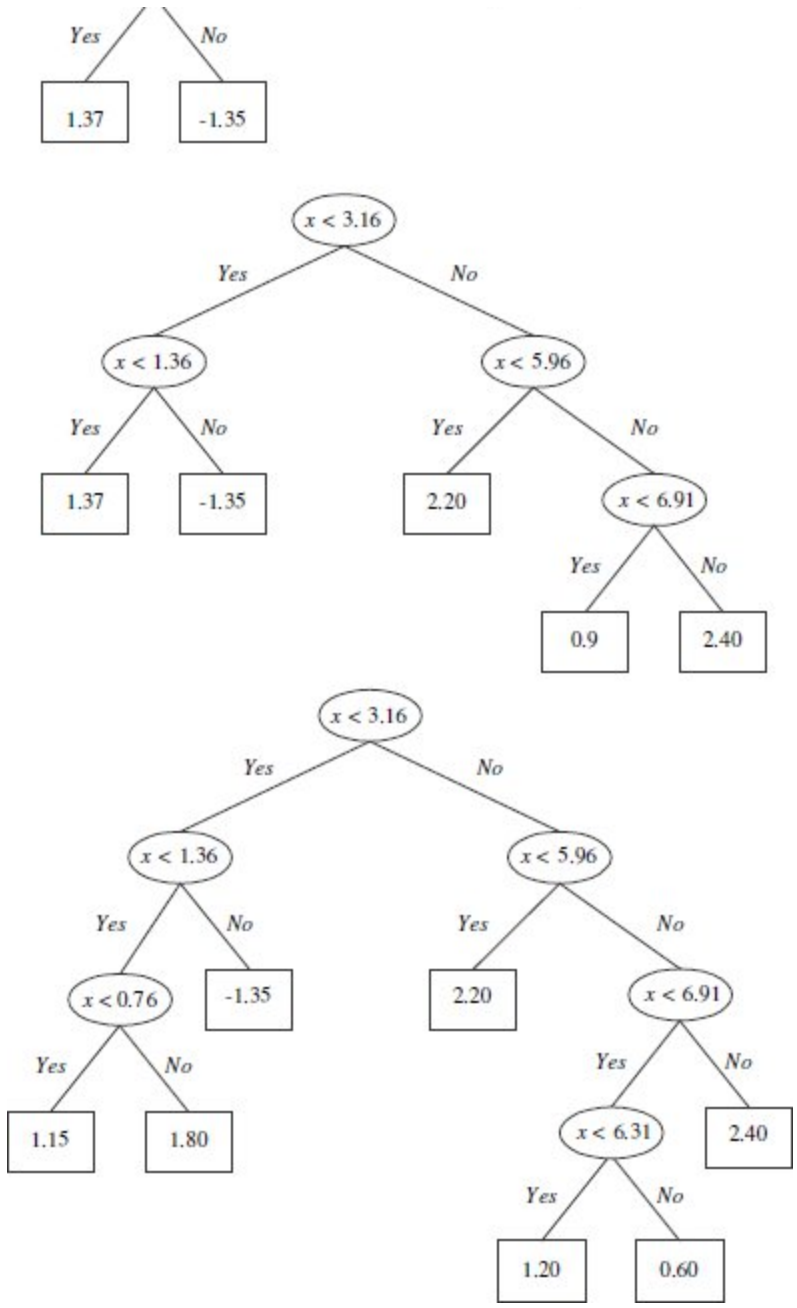


Figure 9.5 Regression trees implementing the smooths of figure 9.4 for various values of θ_r.

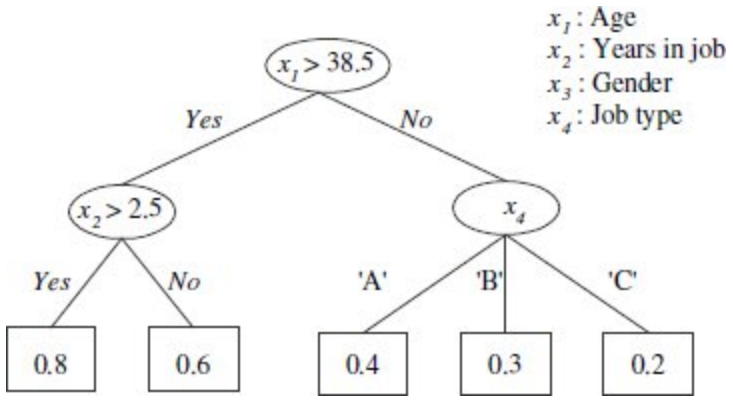


Figure 9.6 Example of a (hypothetical) decision tree. Each path from the root to a leaf can be written down as a conjunctive rule, composed of conditions defined by the decision nodes on the path.

9.4 Rule Extraction from Trees

A decision tree does its own feature extraction. The univariate tree only uses the necessary variables, and after the tree is built, certain features may not be used at all. We can also say that features closer to the root are more important globally. For example, the decision tree given in figure 9.6 uses x_1, x_2, and x_4, but not x_3. It is possible to use a decision tree for feature extraction: we build a tree and then take only those features used by the tree as inputs to another learning method.

Another main advantage of decision trees is ***interpretability***: The decision nodes carry conditions that are simple to understand. Each path from the root to a leaf corresponds to one conjunction of tests, as all those conditions should be satisfied to reach to the leaf. These paths together can be written down as a set of ***IF-THEN rules***, called a *rule base*. One such method is *C4.5Rules* (Quinlan 1993).

For example, the decision tree of figure 9.6 can be written down as the following set of rules:

R1: IF (age > 38.5) AND (years-in-job > 2.5) THEN $y = 0.8$
R2: IF (age > 38.5) AND (years-in-job ≤ 2.5) THEN $y = 0.6$
R3: IF (age ≤ 38.5) AND (job-type = 'A') THEN $y = 0.4$
R4: IF (age ≤ 38.5) AND (job-type = 'B') THEN $y = 0.3$
R5: IF (age ≤ 38.5) AND (job-type = 'C') THEN $y = 0.2$

Such a rule base allows ***knowledge extraction***; it can be easily understood and allows experts to verify the model learned from data. For each rule, one can also calculate the percentage of training data covered by the rule, namely, ***rule support***. The rules reflect the main characteristics of the dataset: They show the important features and split positions. For instance, in this (hypothetical) example, we see that in terms of our purpose (y), people who are thirty-eight years old or less are different from people who are thirty-nine or more years old. And among this latter group, it is the job type that makes them different, whereas in the former group, it is the number of years in a job that is the best discriminating characteristic.

In the case of a classification tree, there may be more than one leaf labeled with the same class. In such a case, these multiple conjunctive expressions corresponding to different paths can be combined as a disjunction (OR). The class region then corresponds to a union of these multiple patches, each patch corresponding to the region defined by one leaf. For example, class C_1 of figure 9.1 is written as

IF $(x_1 \leq w_{10})$ OR $((x_1 > w_{10})$ AND $(x_2 \leq w_{20}))$ THEN C_1

One can also ***prune rules*** for simplification. Pruning a subtree corresponds to pruning terms from a number of rules at the same time. It may be possible to prune a term from one rule without touching other rules. For example, in

the previous rule set, for R3, if we see that all whose `job-type='A'` have outcomes close to 0.4, regardless of `age`, $R3$ can be pruned as

$R3'$: IF (job-type='A') THEN $y = 0.4$

Note that after the rules are pruned, it may not be possible to write them back as a tree anymore.

9.5 Learning Rules from Data

As we have just seen, one way to get IF-THEN rules is to train a decision tree and convert it to rules. Another is to learn the rules directly. ***Rule induction*** works similar to tree induction except that rule induction does a depth-first search and generates one path (rule) at a time, whereas tree induction goes breadth-first and generates all paths simultaneously.

Rules are learned one at a time. Each rule is a conjunction of conditions on discrete or numeric attributes (as in decision trees) and these conditions are added one at a time, to optimize some criterion, for example, minimize entropy. A rule is said to *cover* an example if the example satisfies all the conditions of the rule. Once a rule is grown and pruned, it is added to the rule base and all the training examples covered by the rule are removed from the training set, and the process continues until enough rules are added. This is called ***sequential covering***. There is an outer loop of adding one rule at a time to the rule base and an inner loop of adding one condition at a time to the current rule. These steps are both greedy and do not guarantee optimality. Both loops have a pruning step for better generalization.

One example of a rule induction algorithm is ***Ripper*** (Cohen 1995), based on an earlier algorithm ***Irep*** (Fürnkranz and Widmer 1994). We start with the case of two classes where we talk of positive and negative examples, then later generalize to $K > 2$ classes. Rules are added to explain

positive examples such that if an instance is not covered by any rule, then it is classified as negative. So a rule when it matches is either correct (true positive), or it causes a false positive. The pseudocode of the outer loop of Ripper is given in figure 9.7.

In Ripper, conditions are added to the rule to maximize an information gain measure used in Quinlan's (1990) ***Foil*** algorithm. Let us say we have rule R and R' is the candidate rule after adding a condition. Change in gain is defined as

$$Gain(R', R) = s \cdot \left(\log_2 \frac{N'_+}{N'} - \log_2 \frac{N_+}{N}\right) \tag{9.17}$$

where N is the number of instances that are covered by R and N_+ is the number of true positives in them. N' and N'_+ are similarly defined for R'. s is the number of true positives in R, which are still true positives in R', after adding the condition. In terms of information theory, the change in gain measures the reduction in bits to encode a positive instance.

Conditions are added to a rule until it covers no negative example. Once a rule is grown, it is pruned back by deleting conditions in reverse order, to find the rule that maximizes the ***rule value metric***

$$rvm(R) = \frac{p - n}{p + n} \tag{9.18}$$

where p and n are the number of true and false positives, respectively, on the pruning set, which is one-third of the data, having used two-thirds as the growing set.

```
Ripper(Pos,Neg,k)
    RuleSet ← LearnRuleSet(Pos,Neg)
    For k times
        RuleSet ← OptimizeRuleSet(RuleSet,Pos,Neg)
LearnRuleSet(Pos,Neg)
    RuleSet ← ∅
    DL ← DescLen(RuleSet,Pos,Neg)
    Repeat
        Rule ← LearnRule(Pos,Neg)
```

```
        Rule ← LearnRule(Pos,Neg)
        Add Rule to RuleSet
        DL' ← DescLen(RuleSet,Pos,Neg)
        If DL'>DL+64
          PruneRuleSet(RuleSet,Pos,Neg)
          Return RuleSet
        If DL'<DL DL ← DL'
          Delete instances covered by Rule from Pos and Neg
    Until Pos = ∅
    Return RuleSet
PruneRuleSet(RuleSet,Pos,Neg)
    For each Rule ∈ RuleSet in reverse order
        DL ← DescLen(RuleSet,Pos,Neg)
        DL' ← DescLen(RuleSet-Rule,Pos,Neg)
        IF DL'<DL Delete Rule from RuleSet
    Return RuleSet
OptimizeRuleSet(RuleSet,Pos,Neg)
    For each Rule ∈ RuleSet
        DL0 ← DescLen(RuleSet,Pos,Neg)
        DL1 ← DescLen(RuleSet-Rule+
                ReplaceRule(RuleSet,Pos,Neg),Pos,Neg)
        DL2 ← DescLen(RuleSet-Rule+
                ReviseRule(RuleSet,Rule,Pos,Neg),Pos,Neg)
        If DL1=min(DL0,DL1,DL2)
                Delete Rule from RuleSet and
                        add ReplaceRule(RuleSet,Pos,Neg)
        Else If DL2=min(DL0,DL1,DL2)
                Delete Rule from RuleSet and
                        add ReviseRule(RuleSet,Rule,Pos,Neg)
    Return RuleSet
```

Figure 9.7 Ripper algorithm for learning rules. Only the outer loop is given; the inner loop is similar to adding nodes in a decision tree.

Once a rule is grown and pruned, all positive and negative training examples covered by the rule are removed from the training set. If there are remaining positive examples, rule induction continues. In the case of noise, we may stop early, namely, when a rule does not explain enough number of examples. To measure the worth of a rule, minimum

description length (section 4.8) is used (Quinlan 1995). Typically, we stop if the description of the rule is not shorter than the description of instances it explains. The description length of a rule base is the sum of the description lengths of all the rules in the rule base, plus the description of instances not covered by the rule base. Ripper stops adding rules when the description length of the rule base is more than 64 bits larger than the best description length so far. Once the rule base is learned, we pass over the rules in reverse order to see if they can be removed without increasing the description length.

Rules in the rule base are also optimized after they are learned. Ripper considers two alternatives to a rule: One, called the replacement rule, starts from an empty rule, is grown, and is then pruned. The second, called the revision rule, starts with the rule as it is, is grown, and is then pruned. These two are compared with the original rule, and the shortest of three is added to the rule base. This optimization of the rule base can be done k times, typically twice.

When there are $K > 2$ classes, they are ordered in terms of their prior probabilities such that C_1 has the lowest prior probability and C_K has the highest. Then a sequence of two-class problems are defined such that, first, instances belonging to C_1 are taken as positive examples and instances of all other classes are taken as negative examples. Then, having learned C_1 and all its instances removed, it learns to separate C_2 from $C_3, \ldots, C_K$. This process is repeated until only C_K remains. The empty default rule is then labeled C_K, so that if an instance is not covered by any rule, it will be assigned to C_K.

For a training set of size N, Ripper's complexity is $O(N \log^2 N)$ and is an algorithm that can be used on very large training sets (Dietterich 1997). The rules we learn are ***propositional rules***. More expressive, ***first-order rules*** have variables in

conditions, called *predicates*. A *predicate* is a function that returns true or false depending on the value of its argument. Predicates therefore allow defining relations between the values of attributes, which cannot be done by propositions (Mitchell 1997):

IF Father(y, x) AND Female(y) THEN Daughter(x, y)

Such rules can be seen as programs in a logic programming language, such as Prolog, and learning them from data is called ***inductive logic programming***. One such algorithm is Foil (Quinlan 1990).

Assigning a value to a variable is called ***binding***. A rule matches if there is a set of bindings to the variables existing in the training set. Learning first-order rules is similar to learning propositional rules with an outer loop of adding rules, and an inner loop of adding conditions to a rule, with prunings at the end of each loop. The difference is in the inner loop, where at each step we consider one predicate to add (instead of a proposition) and check the increase in the performance of the rule (Mitchell 1997). To calculate the performance of a rule, we consider all possible bindings of the variables, count the number of positive and negative bindings in the training set, and use, for example, equation 9.17. In this first-order case, we have predicates instead of propositions, so they should be previously defined, and the training set is a set of predicates known to be true.

9.6 Multivariate Trees

In the case of a univariate tree, only one input dimension is used at a split. In a ***multivariate tree***, at a decision node, all input dimensions can be used and thus it is more general. When all inputs are numeric, a binary linear multivariate node is defined as

14 *Kernel Machines*

Kernel machines are maximum margin methods that allow the model to be written as a sum of the influences of a subset of the training instances. These influences are given by application-specific similarity kernels, and we discuss "kernelized" classification, regression, ranking, outlier detection and dimensionality reduction, and how to choose and use kernels.

14.1 Introduction

We now discuss a different approach for linear classification and regression. We should not be surprised to have so many different methods even for the simple case of a linear model. Each learning algorithm has a different inductive bias, makes different assumptions, and defines a different objective function and thus may find a different linear model.

The model that we will discuss in this chapter, called the *support vector machine* (SVM), and later generalized under the name *kernel machine*, has been popular in recent years for a number of reasons:

1. It is a discriminant-based method and uses Vapnik's principle to never solve a more complex problem as a first step before the actual problem (Vapnik 1995). For example, in classification, when the task is to

learn the discriminant, it is not necessary to estimate the class densities $p(\boldsymbol{x}|C_i)$ or the exact posterior probability values $P(C_i|\boldsymbol{x})$; we only need to estimate where the class boundaries lie, that is, $\boldsymbol{x}$ where $P(C_i|\boldsymbol{x}) = P(C_j|\boldsymbol{x})$. Similarly, for outlier detection, we do not need to estimate the full density $p(\boldsymbol{x})$; we only need to find the boundary separating those $\boldsymbol{x}$ that have low $p(\boldsymbol{x})$, that is, $\boldsymbol{x}$ where $p(\boldsymbol{x}) < \theta$, for some threshold $\theta \in (0, 1)$.

2. After training, the parameter of the linear model, the weight vector, can be written down in terms of a subset of the training set, which are the so-called *support vectors*. In classification, these are the cases that are close to the boundary and as such, knowing them allows knowledge extraction: Those are the uncertain or erroneous cases that lie in the vicinity of the boundary between two classes. Their number gives us an estimate of the generalization error, and, as we see below, being able to write the model parameter in terms of a set of instances allows kernelization.
3. As we will see shortly, the output is written as a sum of the influences of support vectors and these are given by *kernel functions* that are application-specific measures of similarity between data instances. Previously, we talked about nonlinear basis functions allowing us to map the input to another space where a linear (smooth) solution is possible; the kernel function uses the same idea.
4. Typically in most learning algorithms, data points are represented as vectors, and either dot product (as in the multilayer perceptrons) or Euclidean distance (as in radial basis function networks) is used. A kernel

function allows us to go beyond that. For example, G_1 and G_2 may be two graphs and $K(G_1, G_2)$ may correspond to the number of shared paths, which we can calculate without needing to represent G_1 or G_2 explicitly as vectors.

5. Kernel-based algorithms are formulated as convex optimization problems and there is a single optimum. Though finding it may still require an iterative process, we have a much easier task than what we have with multilayer perceptrons, for example. Of course, this does not mean that we do not have any hyperparameters for model selection; we do—any method needs them, to match the algorithm to the data at hand.

We start our discussion with the case of classification, and then generalize to regression, ranking, outlier (novelty) detection, and then dimensionality reduction. We see that in all cases basically we have the similar quadratic program template to maximize the separability, or *margin*, of instances subject to a constraint of the smoothness of solution. Solving for it, we get the support vectors. The kernel function defines the space according to its notion of similarity and a kernel function is good if we have better separation in its corresponding space.

14.2 Optimal Separating Hyperplane

Let us start again with two classes and use labels −1/ + 1 for the two classes. The sample is $\mathcal{X} = \{\boldsymbol{x}^t, r^t\}$ where $r^t = +1$ if $\boldsymbol{x}^t \in C_1$ and $r^t = -1$ if $\boldsymbol{x}^t \in C_2$. We would like to find $\boldsymbol{w}$ and w_0 such that

$$\boldsymbol{w}^T\boldsymbol{x}^t + w_0 \geq +1 \quad \text{for} \quad r^t = +1$$

$$\boldsymbol{w}^T\boldsymbol{x}^t + w_0 \le -1 \quad \text{for} \quad r^t = -1$$

which can be rewritten as

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \ge +1 \tag{14.1}$$

Note that we do not simply require

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \ge 0$$

Not only do we want the instances to be on the right side of the hyperplane, but we also want them some distance away, for better generalization. The distance from the hyperplane to the instances closest to it on either side is called the ***margin***, which we want to maximize for best generalization.

Very early on, in section 2.1, we talked about the concept of the margin when we were talking about fitting a rectangle, and we said that it is better to take a rectangle halfway between S and G, to get a breathing space. This is so that in case noise shifts a test instance slightly, it will still be on the right side of the boundary.

Similarly, now that we are using the hypothesis class of lines, the ***optimal separating hyperplane*** is the one that maximizes the margin.

We remember from section 10.3 that the distance of $\boldsymbol{x}^t$ to the discriminant is

$$\frac{|\boldsymbol{w}^T\boldsymbol{x}^t + w_0|}{\|\boldsymbol{w}\|}$$

which, when $r^t \in \{-1, +1\}$, can be written as

$$\frac{r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0)}{\|\boldsymbol{w}\|}$$

and we would like this to be at least some value ρ:

$$\frac{r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0)}{\|\boldsymbol{w}\|} \geq \rho, \forall t \tag{14.2}$$

We would like to maximize ρ but there are an infinite number of solutions that we can get by scaling $\boldsymbol{w}$ and for a unique solution, we fix $\rho\|\boldsymbol{w}\| = 1$ and thus, to maximize the margin, we minimize $\|\boldsymbol{w}\|$. The task can therefore be defined (see Cortes and Vapnik 1995; Vapnik 1995) as to

$$\min \frac{1}{2}\|\boldsymbol{w}\|^2 \text{ subject to } r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \geq +1, \forall t \tag{14.3}$$

This is a standard quadratic programming problem, whose complexity depends on d, and it can be solved directly to find $\boldsymbol{w}$ and w_0. Then, on both sides of the hyperplane, there will be instances that are $1/\|\boldsymbol{w}\|$ away from the hyperplane and the total margin will be $2/\|\boldsymbol{w}\|$.

We saw in section 10.2 that if the problem is not linearly separable, instead of fitting a nonlinear function, one trick is to map the problem to a new space by using nonlinear basis functions. It is generally the case that this new space has many more dimensions than the original space, and, in such a case, we are interested in a method whose complexity does not depend on the input dimensionality.

In finding the optimal hyperplane, we can convert the optimization problem to a form whose complexity depends on N, the number of training instances, and not on d. Another advantage of this new formulation is that it will allow us to rewrite the basis functions in terms of kernel functions, as we will see in section 14.5.

To get the new formulation, we first write equation 14.3 as an unconstrained problem using Lagrange multipliers α^t:

$$\begin{aligned} L_p &= \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_{t=1}^{N} \alpha^t[r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) - 1] \\ &= \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_t \alpha^t r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) + \sum_t \alpha^t \end{aligned} \tag{14.4}$$

This should be minimized with respect to $\boldsymbol{w}$, w_0 and maximized with respect to $\alpha^t \geq 0$. The saddle point gives the solution.

This is a convex quadratic optimization problem because the main term is convex and the linear constraints are also convex. Therefore, we can equivalently solve the dual problem, making use of the Karush-Kuhn-Tucker conditions. The dual is to *maximize* L_p with respect to α^t, subject to the constraints that the gradient of L_p with respect to $\boldsymbol{w}$ and w_0 are 0 and also that $\alpha^t \geq 0$:

$$\frac{\partial L_p}{\partial \boldsymbol{w}} = 0 \quad \Rightarrow \quad \boldsymbol{w} = \sum_t \alpha^t r^t \boldsymbol{x}^t \tag{14.5}$$

$$\frac{\partial L_p}{\partial w_0} = 0 \quad \Rightarrow \quad \sum_t \alpha^t r^t = 0 \tag{14.6}$$

Plugging these into equation 14.4, we get the dual

$$\begin{aligned} L_d &= \frac{1}{2}(\boldsymbol{w}^T\boldsymbol{w}) - \boldsymbol{w}^T \sum_t \alpha^t r^t \boldsymbol{x}^t - w_0 \sum_t \alpha^t r^t + \sum_t \alpha^t \\ &= -\frac{1}{2}(\boldsymbol{w}^T\boldsymbol{w}) + \sum_t \alpha^t \\ &= -\frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s (\boldsymbol{x}^t)^T \boldsymbol{x}^s + \sum_t \alpha^t \end{aligned} \tag{14.7}$$

which we maximize with respect to α^t only, subject to the constraints

$$\sum_t \alpha^t r^t = 0, \text{ and } \alpha^t \geq 0, \forall t$$

This can be solved using quadratic programming methods. The size of the dual depends on N, sample size, and not on d, the input dimensionality. The upper bound for time complexity is $O(N^3)$, and the upper bound for space complexity is $O(N^2)$.

Once we solve for α^t, we see that though there are N of them, most vanish with $\alpha^t = 0$ and only a small percentage have $\alpha^t > 0$. The set of $\boldsymbol{x}^t$ whose $\alpha^t > 0$ are the *support vectors*, and as we see in equation 14.5, $\boldsymbol{w}$ is written as the weighted sum of these training instances that are selected as the support vectors. These are the $\boldsymbol{x}^t$ that satisfy

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) = 1$$

and lie on the margin. We can use this fact to calculate w_0 from any support vector as

$$w_0 = r^t - \boldsymbol{w}^T\boldsymbol{x}^t \tag{14.8}$$

For numerical stability, it is advised that this be done for all support vectors and an average be taken. The discriminant thus found is called the ***support vector machine*** (SVM) (see figure 14.1).

The majority of the α^t are 0, for which $r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) > 1$. These are the $\boldsymbol{x}^t$ that lie more than sufficiently away from the discriminant, and they have no effect on the hyperplane. The instances that are not support vectors carry no information; even if any subset of them are removed, we would still get the same solution. From this perspective, the SVM algorithm can be likened to the condensed nearest neighbor algorithm (section 8.5), which stores only the instances neighboring (and hence constraining) the class discriminant.

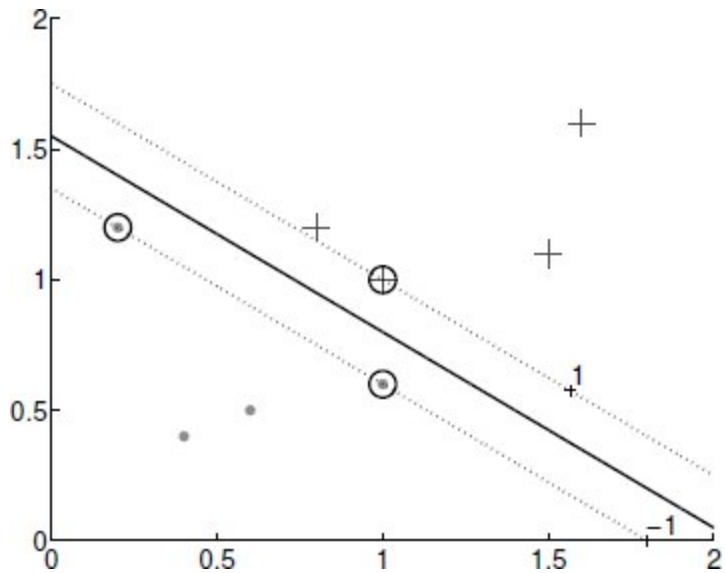


Figure 14.1 For a two-class problem where the instances of the classes are shown by plus signs and dots, the thick line is the boundary and the dashed lines define the margins on either side. Circled instances are the support vectors.

Being a discriminant-based method, the SVM cares only about the instances close to the boundary and discards those that lie in the interior. Using this idea, it is possible to use a simpler classifier before the SVM to filter out a large portion of such instances, thereby decreasing the complexity of the optimization step of the SVM (exercise 1).

During testing, we do not enforce a margin. We calculate $g(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + w_0$, and choose according to the sign of $g(\boldsymbol{x})$:

Choose C_1 if $g(\boldsymbol{x}) > 0$ and C_2 otherwise

14.3 The Nonseparable Case: Soft Margin Hyperplane

If the data is not linearly separable, the algorithm we discussed earlier will not work. In such a case, if the two

classes are not linearly separable such that there is no hyperplane to separate them, we look for the one that incurs the least error. We define ***slack variables***, $\xi^t \geq 0$, which store the deviation from the margin. There are two types of deviation: An instance may lie on the wrong side of the hyperplane and be misclassified. Or, it may be on the right side but may lie in the margin, namely, not sufficiently away from the hyperplane. Relaxing equation 14.1, we require

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \geq 1 - \xi^t \tag{14.9}$$

If $\xi^t = 0$, there is no problem with $\boldsymbol{x}^t$. If $0 < \xi^t < 1$, $\boldsymbol{x}^t$ is correctly classified but in the margin. If $\xi^t \geq 1$, $\boldsymbol{x}^t$ is misclassified (see figure 14.2). The number of misclassifications is $\#\{\xi^t > 1\}$, and the number of nonseparable points is $\#\{\xi_t > 0\}$. We define ***soft error*** as

$$\sum_t \xi^t$$

and add this as a penalty term:

$$L_p = \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_t \xi^t \tag{14.10}$$

subject to the constraint of equation 14.9. C is the penalty factor as in any regularization scheme trading off complexity, as measured by the L_2 norm of the weight vector (similar to weight decay in multilayer perceptrons; see section 12.4.2), and data misfit, as measured by the number of nonseparable points. Note that we are penalizing not only the misclassified points but also the ones in the margin for better generalization, though these latter would be correctly classified during testing.

Adding the constraints, the Lagrangian of equation 14.4 then becomes

$$L_p = \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_t \xi^t - \sum_t \alpha^t[r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) - 1 + \xi^t] - \sum_t \mu^t\xi^t \tag{14.11}$$

where μ_t are the new Lagrange parameters to guarantee the positivity of ξ^t. When we take the derivatives with respect to the parameters and set them to 0, we get

$$\frac{\partial L_p}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_t \alpha^t r^t \boldsymbol{x}^t = 0 \Rightarrow \boldsymbol{w} = \sum_t \alpha^t r^t \boldsymbol{x}^t \tag{14.12}$$

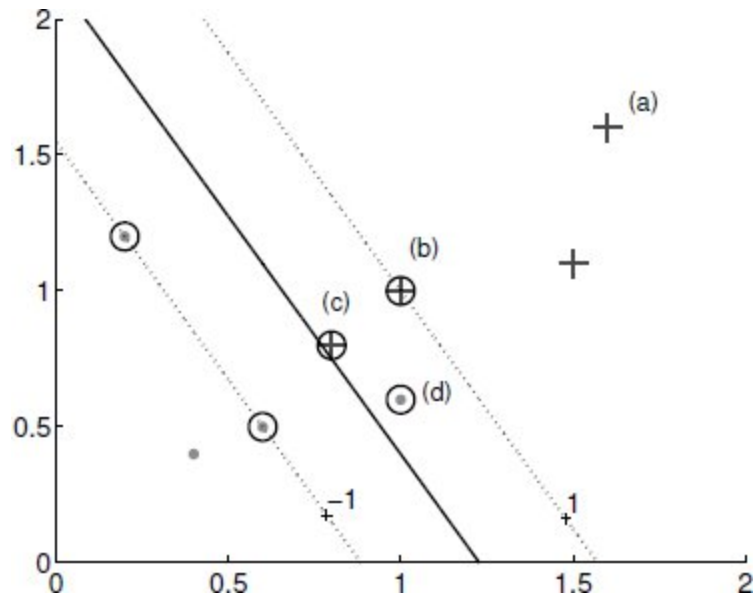


Figure 14.2 In classifying an instance, there are four possible cases: In (a), the instance is on the correct side and far away from the margin; $r^t g(\boldsymbol{x}^t) > 1$, $\xi^t = 0$. In (b), $\xi^t = 0$; it is on the right side and on the margin. In (c), $\xi^t = 1 - g(\boldsymbol{x}^t)$, $0 < \xi < 1$; it is on the right side but is in the margin and not sufficiently away. In (d), $\xi^t = 1 + g(\boldsymbol{x}^t) > 1$; it is on the wrong side—this is a misclassification. All cases except (a) are support vectors. In terms of the dual variable, in (a), $\alpha^t = 0$; in (b), $\alpha^t < C$; in (c) and (d), $\alpha^t = C$.

$$(14.13) \quad \frac{\partial L_p}{\partial w_0} = \sum_t \alpha^t r^t = 0$$

$$(14.14) \quad \frac{\partial L_p}{\partial \xi^t} = C - \alpha^t - \mu^t = 0$$

Since $\mu^t \geq 0$, this last implies that $0 \leq \alpha^t \leq C$. Plugging these into equation 14.11, we get the dual that we maximize with respect to α^t:

$$(14.15) \quad L_d = \sum_t \alpha^t - \frac{1}{2} \sum_t \sum_s \alpha^t \alpha^s r^t r^s (\mathbf{x}^t)^T \mathbf{x}^s$$

subject to

$$\sum_t \alpha^t r^t = 0 \text{ and } 0 \leq \alpha^t \leq C, \forall t$$

Solving this, we see that as in the separable case, instances that lie on the correct side of the boundary with sufficient margin vanish with their $\alpha^t = 0$ (see figure 14.2). The support vectors have their $\alpha^t > 0$ and they define $\mathbf{w}$, as given in equation 14.12. Of these, those whose $\alpha^t < C$ are the ones that are on the margin, and we can use them to calculate w_0; they have $\xi^t = 0$ and satisfy $r^t(\mathbf{w}^T\mathbf{x}^t + w_0) = 1$. Again, it is better to take an average over these w_0 estimates. Those instances that are in the margin or misclassified have their $\alpha^t = C$.

The nonseparable instances that we store as support vectors are the instances that we would have trouble correctly classifying if they were not in the training set; they would either be misclassified or classified correctly but not with enough confidence. We can say that the number of support vectors is an upper-bound estimate for the expected number of errors. And, actually, Vapnik (1995) has shown that the expected test error rate is

$$E_N[P(error)] \leq \frac{E_N[\text{\# of support vectors}]}{N}$$

where $E_N[\cdot]$ denotes expectation over training sets of size N. The nice implication of this is that it shows that the error rate depends on the number of support vectors and not on the input dimensionality.

Equation 14.9 implies that we define error if the instance is on the wrong side or if the margin is less than 1. This is called the ***hinge loss***. If $y^t = \mathbf{w}^T\mathbf{x}^t + w_0$ is the output and r^t is the desired output, hinge loss is defined as

$$L_{hinge}(y^t, r^t) = \begin{cases} 0 & \text{if } y^t r^t \geq 1 \\ 1 - y^t r^t & \text{otherwise} \end{cases} \tag{14.16}$$

In figure 14.3, we compare hinge loss with 0/1 loss, squared error, and cross-entropy. We see that unlike 0/1 loss, hinge loss also penalizes instances in the margin even though they may be on the correct side, and the loss increases linearly as the instance moves away on the wrong side. This is different from the squared loss that therefore is not as robust as the hinge loss. We see that cross-entropy minimized in logistic discrimination (section 10.7) or by the linear perceptron (section 11.3) is a good continuous approximation to the hinge loss.

C of equation 14.10 is the regularization parameter fine-tuned using cross-validation. It defines the trade-off between margin maximization and error minimization: If it is too large, we have a high penalty for nonseparable points, and we may store many support vectors and overfit. If it is too small, we may find too simple solutions that underfit. Typically, one chooses from $[10^{-6}, 10^{-5}, \ldots, 10^{+5}, 10^{+6}]$ in the log scale by looking at the accuracy on a validation set.

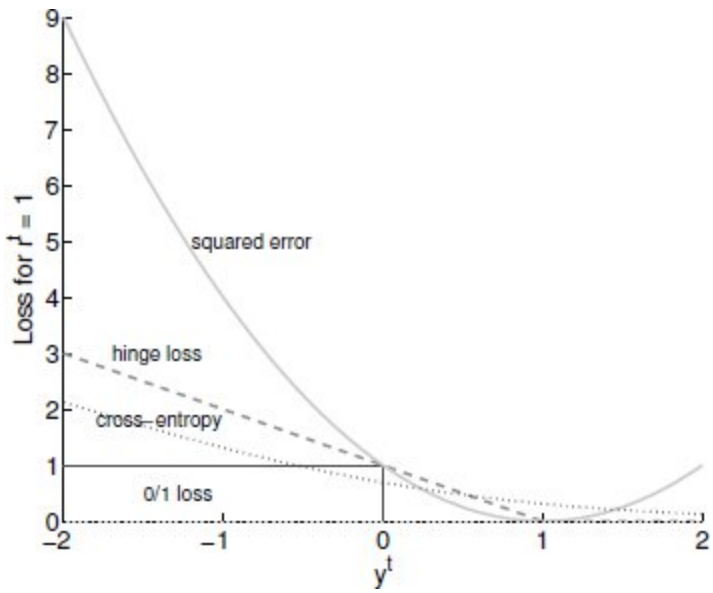


Figure 14.3 Comparison of different loss functions for $r^t = 1$: 0/1 loss is 0 if $y^t = 1$, 1 otherwise. Hinge loss is 0 if $y^t > 1$, $1 - y^t$ otherwise. Squared error is $(1 - y^t)^2$. Cross-entropy is $\log(1/(1 + \exp(-y^t)))$.

14.4 ν-SVM

There is another, equivalent formulation of the soft margin hyperplane that uses a parameter $\nu \in [0, 1]$ instead of C (Schölkopf et al. 2000). The objective function is

$$\min \frac{1}{2}\|\mathbf{w}\|^2 - \nu\rho + \frac{1}{N}\sum_t \xi^t \tag{14.17}$$

subject to

$$r^t(\mathbf{w}^T\mathbf{x}^t + w_0) \geq \rho - \xi^t,\ \xi^t \geq 0,\ \rho \geq 0 \tag{14.18}$$

ρ is a new parameter that is a variable of the optimization problem and scales the margin: The margin is now $2\rho/\|\mathbf{w}\|$. ν has been shown to be a lower bound on the fraction of

support vectors and an upper bound on the fraction of instances having margin errors $(\sum_t \#\{\xi^t > 0\})$. The dual is

$$L_d = -\frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s (\mathbf{x}^t)^T \mathbf{x}^s \tag{14.19}$$

subject to

$$\sum_t \alpha^t r^t = 0,\ 0 \le \alpha^t \le \frac{1}{N},\ \sum_t \alpha^t \ge \nu$$

When we compare equation 14.19 with equation 14.15, we see that the term $\sum_t \alpha^t$ no longer appears in the objective function but is now a constraint. By playing with ν, we can control the fraction of support vectors, and this is advocated to be more intuitive than playing with C.

14.5 Kernel Trick

Section 10.2 demonstrated that if the problem is nonlinear, instead of trying to fit a nonlinear model, we can map the problem to a new space by doing a nonlinear transformation using suitably chosen basis functions and then use a linear model in this new space. The linear model in the new space corresponds to a nonlinear model in the original space. This approach can be used in both classification and regression problems, and in the special case of classification, it can be used with any scheme. In the particular case of support vector machines, it leads to certain simplifications that we now discuss.

Let us say we have the new dimensions calculated through the basis functions

$$\mathbf{z} = \boldsymbol{\phi}(\mathbf{x}) \text{ where } z_j = \phi_j(\mathbf{x}),\ j = 1, \ldots, k$$

mapping from the d-dimensional $\mathbf{x}$ space to the k-dimensional $\mathbf{z}$ space where we write the discriminant as

$$
\begin{aligned}
g(\mathbf{z}) &= \mathbf{w}^T \mathbf{z} \\
g(\mathbf{x}) &= \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) \\
&= \sum_{j-1}^{k} w_j \phi_j(\mathbf{x}) \qquad (14.20)
\end{aligned}
$$

where we do not use a separate w_0; we assume that $z_1 = \boldsymbol{\phi}_1(\mathbf{x}) \equiv 1$. Generally, k is much larger than d and k may also be larger than N, and there lies the advantage of using the dual form whose complexity depends on N, whereas if we used the primal it would depend on k. We also use the more general case of the soft margin hyperplane here because we have no guarantee that the problem is linearly separable in this new space.

The problem is the same

$$L_p = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_t \xi^t \qquad (14.21)$$

except that now the constraints are defined in the new space

$$r^t \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^t) \geq 1 - \xi^t \qquad (14.22)$$

The Lagrangian is

$$L_p = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_t \xi^t - \sum_t \alpha^t \left[r^t \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}^t) - 1 + \xi^t\right] - \sum_t \mu^t \xi^t \qquad (14.23)$$

When we take the derivatives with respect to the parameters and set them to 0, we get

$$\frac{\partial L_p}{\partial \mathbf{w}} = \mathbf{w} = \sum_t \alpha^t r^t \boldsymbol{\phi}(\mathbf{x}^t) \qquad (14.24)$$

$$\frac{\partial L_p}{\partial \xi^t} = C - \alpha^t - \mu^t = 0 \qquad (14.25)$$

The dual is now

$$L_d = \sum_t \alpha^t - \frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s \boldsymbol{\phi}(\mathbf{x}^t)^T \boldsymbol{\phi}(\mathbf{x}^s) \qquad (14.26)$$

subject to

$$\sum_t \alpha^t r^t = 0 \text{ and } 0 \le \alpha^t \le C, \forall t$$

The idea in *kernel machines* is to replace the inner product of basis functions, $\boldsymbol{\phi}(\boldsymbol{x}^t)^T\boldsymbol{\phi}(\boldsymbol{x}^s)$, by a ***kernel function***, $K(\boldsymbol{x}^t, \boldsymbol{x}^s)$, between instances in the original input space. So instead of mapping two instances $\boldsymbol{x}^t$ and $\boldsymbol{x}^s$ to the $\boldsymbol{z}$-space and doing a dot product there, we directly apply the kernel function in the original space.

$$L_d = \sum_t \alpha^t - \frac{1}{2}\sum_t\sum_s \alpha^t\alpha^s r^t r^s K(\boldsymbol{x}^t, \boldsymbol{x}^s) \tag{14.27}$$

The kernel function also shows up in the discriminant

$$\begin{aligned} g(\boldsymbol{x}) &= \boldsymbol{w}^T\boldsymbol{\phi}(\boldsymbol{x}) = \sum_t \alpha^t r^t \boldsymbol{\phi}(\boldsymbol{x}^t)^T\boldsymbol{\phi}(\boldsymbol{x}) \\ &= \sum_t \alpha^t r^t K(\boldsymbol{x}^t, \boldsymbol{x}) \end{aligned} \tag{14.28}$$

This implies that if we have the kernel function, we do not need to map it to the new space at all. Actually, for any valid kernel, there does exist a corresponding mapping function, but it may be much simpler to use $K(\boldsymbol{x}^t, \boldsymbol{x})$ rather than calculating $\boldsymbol{\phi}(\boldsymbol{x}^t)$, $\boldsymbol{\phi}(\boldsymbol{x})$ and taking the dot product. Many algorithms have been ***kernelized***, as we will see in later sections, and that is why we have the name "kernel machines."

The matrix of kernel values, $\mathbf{K}$, where $\mathbf{K}_{ts} = K(\boldsymbol{x}^t, \boldsymbol{x}^s)$, is called the ***Gram matrix***, which should be symmetric and positive semidefinite. Recently, it has become standard practice in sharing datasets to have available only the $\mathbf{K}$ matrices without providing $\boldsymbol{x}^t$ or $\boldsymbol{\phi}(\boldsymbol{x}^t)$. Especially in bioinformatics or natural language processing applications where $\boldsymbol{x}$ (or $\boldsymbol{\phi}(\boldsymbol{x})$) has hundreds or thousands of dimensions,

storing/downloading the $N \times N$ matrix is much cheaper (Vert, Tsuda, and Schölkopf 2004); this, however, implies that we can use only those available for training/testing and cannot use the trained model to make predictions outside this dataset.

14.6 Vectorial Kernels

The most popular, general-purpose kernel functions are

- *polynomials* of degree q:

  $$K(\boldsymbol{x}^t, \boldsymbol{x}) = (\boldsymbol{x}^T \boldsymbol{x}^t + 1)^q \tag{14.29}$$

  where q is selected by the user. For example, when $q = 2$ and $d = 2$,

  $$\begin{aligned} K(\boldsymbol{x}, \boldsymbol{y}) &= (\boldsymbol{x}^T \boldsymbol{y} + 1)^2 \\ &= (x_1 y_1 + x_2 y_2 + 1)^2 \\ &= 1 + 2x_1 y_1 + 2x_2 y_2 + 2x_1 x_2 y_1 y_2 + x_1^2 y_1^2 + x_2^2 y_2^2 \end{aligned}$$

  corresponds to the inner product of the basis function (Cherkassky and Mulier 1998):

  $$\phi(\boldsymbol{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1 x_2, x_1^2, x_2^2]^T$$

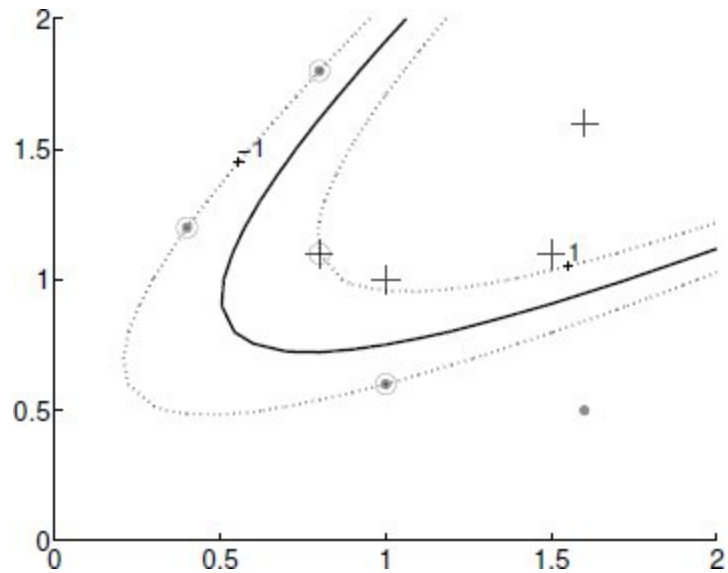


Figure 14.4 The discriminant and margins found by a polynomial kernel of degree 2. Circled instances are the support vectors.

An example is given in figure 14.4. When $q = 1$, we have the *linear kernel* that corresponds to the original formulation.

- *radial-basis functions*:

$$K(\boldsymbol{x}^t, \boldsymbol{x}) = \exp\left[-\frac{\|\boldsymbol{x}^t - \boldsymbol{x}\|^2}{2s^2}\right] \tag{14.30}$$

defines a spherical kernel as in Parzen windows (chapter 8) where $\boldsymbol{x}^t$ is the center and s, supplied by the user, defines the radius. This is also similar to radial basis functions that we discuss in chapter 13.

An example is shown in figure 14.5 where we see that larger spreads smooth the boundary; the best value is found by cross-validation. Note that when there are two parameters to be optimized using cross-validation, for example, here C and s^2, one should do a grid (factorial)

search in the two dimensions; we will discuss methods for searching the best combination of such factors in section 20.2.

One can have a Mahalanobis kernel, generalizing from the Euclidean distance:

$$K(\boldsymbol{x}^t, \boldsymbol{x}) = \exp\left[-\frac{1}{2}(\boldsymbol{x}^t - \boldsymbol{x})^T \mathbf{S}^{-1}(\boldsymbol{x}^t - \boldsymbol{x})\right] \tag{14.31}$$

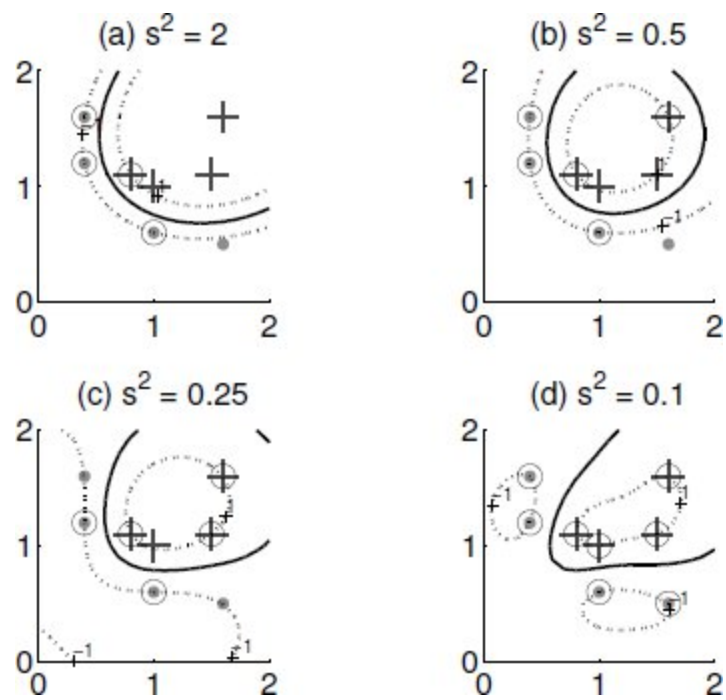


Figure 14.5 The boundary and margins found by the Gaussian kernel with different spread values, s^2. We get smoother boundaries with larger spreads.

where $\mathbf{S}$ is a covariance matrix. Or, in the most general case,

$$K(\boldsymbol{x}^t, \boldsymbol{x}) = \exp\left[-\frac{\mathcal{D}(\boldsymbol{x}^t, \boldsymbol{x})}{2s^2}\right] \tag{14.32}$$

for some distance function $\mathcal{D}(\boldsymbol{x}^t, \boldsymbol{x})$.

- *sigmoidal functions*:

$$K(\boldsymbol{x}^t, \boldsymbol{x}) = \tanh(2\boldsymbol{x}^T\boldsymbol{x}^t + 1) \tag{14.33}$$

  where tanh(·) has the same shape with sigmoid, except that it ranges between −1 and +1. This is similar to multilayer perceptrons that we discussed in chapter 11.

14.7 Defining Kernels

It is also possible to define application-specific kernels. Kernels are generally considered to be measures of similarity in the sense that $K(\boldsymbol{x}, \boldsymbol{y})$ takes a larger value as $\boldsymbol{x}$ and $\boldsymbol{y}$ are more "similar," from the point of view of the application. This implies that any prior knowledge we have regarding the application can be provided to the learner through appropriately defined kernels—"kernel engineering"—and such use of kernels can be seen as another example of a "hint" (section 12.4.1).

There are string kernels, tree kernels, graph kernels, and so on (Vert, Tsuda, and Schölkopf 2004), depending on how we represent the data and how we measure similarity in that representation.

For example, given two documents, the number of words appearing in both may be a kernel. Let us say D_1 and D_2 are two documents and one possible representation is called ***bag of words*** where we predefine M words relevant for the application, and we define $\boldsymbol{\phi}(D_1)$ as the M-dimensional binary vector whose dimension i is 1 if word i appears in D_1 and is 0 otherwise. Then, $\boldsymbol{\phi}(D_1)^T \boldsymbol{\phi}(D_2)$ counts the number of shared words. Here, we see that if we directly define and implement $K(D_1, D_2)$ as the number of shared words, we do not need to preselect M words and can use just any word in the vocabulary (of course, after discarding uninformative words like "of," "and," etc.) and we would not need to generate the

bag-of-words representation explicitly and it would be as if we allowed M to be as large as we want.

Sometimes—for example, in bioinformatics applications—we can calculate a *similarity score* between two objects, which may not necessarily be positive semidefinite. Given two strings (of genes), a kernel measures the ***edit distance***, namely, how many operations (insertions, deletions, substitutions) it takes to convert one string into another; this is also called ***alignment***. In such a case, a trick is to define a set of M templates and represent an object as the M-dimensional vector of scores to all the templates. That is, if $\boldsymbol{m}_i$, $i = 1, \ldots, M$ are the templates and $s(\boldsymbol{x}^t, \boldsymbol{m}_i)$ is the score between $\boldsymbol{x}^t$ and $\boldsymbol{m}_i$, then we define

$$\boldsymbol{\phi}(\boldsymbol{x}^t) = [s(\boldsymbol{x}^t, \boldsymbol{m}_1), s(\boldsymbol{x}^t, \boldsymbol{m}_2), \ldots, s(\boldsymbol{x}^t, \boldsymbol{m}_M)]^T$$

and we define the ***empirical kernel map*** as

$$K(\boldsymbol{x}^t, \boldsymbol{x}^s) = \boldsymbol{\phi}(\boldsymbol{x}^t)^T \boldsymbol{\phi}(\boldsymbol{x}^s)$$

which is a valid kernel.

Sometimes, we have a binary score function; for example, two proteins may interact or not, and we want to be able to generalize from this to scores for two arbitrary instances. In such a case, a trick is to define a graph where the nodes are the instances and two nodes are linked if they interact, that is, if the binary score returns 1. Then we say that two nodes that are not immediately linked are "similar" if the path between them is short or if they are connected by many paths. This converts pairwise local interactions to a global similarity measure, rather like defining a geodesic distance used in Isomap (section 6.10), and it is called the ***diffusion kernel***.

If $p(\boldsymbol{x})$ is a probability density, then

$$K(\boldsymbol{x}^t, \boldsymbol{x}) = p(\boldsymbol{x}^t)p(\boldsymbol{x})$$

is a valid kernel. This is used when $p(\boldsymbol{x})$ is a generative model for $\boldsymbol{x}$ measuring how likely it is that we see $\boldsymbol{x}$. For example, if $\boldsymbol{x}$ is a sequence, $p(\boldsymbol{x})$ can be a hidden Markov model (chapter 16). With this kernel, $K(\boldsymbol{x}^t, \boldsymbol{x})$ will take a high value if both $\boldsymbol{x}^t$ and $\boldsymbol{x}$ are likely to have been generated by the same model. It is also possible to parametrize the generative model as $p(\boldsymbol{x}|\theta)$ and learn θ from data; this is called the ***Fisher kernel*** (Jaakkola and Haussler 1999).

14.8 Multiple Kernel Learning

It is possible to construct new kernels by combining simpler kernels. If $K_1(\boldsymbol{x}, \boldsymbol{y})$ and $K_2(\boldsymbol{x}, \boldsymbol{y})$ are valid kernels and c a constant, then

$$K(\boldsymbol{x}, \boldsymbol{y}) = \begin{cases} cK_1(\boldsymbol{x}, \boldsymbol{y}) \\ K_1(\boldsymbol{x}, \boldsymbol{y}) + K_2(\boldsymbol{x}, \boldsymbol{y}) \\ K_1(\boldsymbol{x}, \boldsymbol{y}) \cdot K_2(\boldsymbol{x}, \boldsymbol{y}) \end{cases} \tag{14.34}$$

are also valid.

Different kernels may also be using different subsets of $\boldsymbol{x}$. We can therefore see combining kernels as another way to fuse information from different sources where each kernel measures similarity according to its domain. When we have input from two representations A and B

$$\begin{aligned} K_A(\boldsymbol{x}_A, \boldsymbol{y}_A) + K_B(\boldsymbol{x}_B, \boldsymbol{y}_B) &= \boldsymbol{\phi}_A(\boldsymbol{x}_A)^T \boldsymbol{\phi}_A(\boldsymbol{y}_A) + \boldsymbol{\phi}_B(\boldsymbol{x}_B)^T \boldsymbol{\phi}_B(\boldsymbol{y}_B) \\ &= \boldsymbol{\phi}(\boldsymbol{x})^T \boldsymbol{\phi}(\boldsymbol{y}) \\ &= K(\boldsymbol{x}, \boldsymbol{y}) \end{aligned} \tag{14.35}$$

where $\boldsymbol{x} = [\boldsymbol{x}_A, \boldsymbol{x}_B]$ is the concatenation of the two representations. That is, taking a sum of two kernels corresponds to doing a dot product in the concatenated feature vectors. One can generalize to a number of kernels

$$(14.36) \quad K(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i=1}^{m} K_i(\boldsymbol{x}, \boldsymbol{y})$$

which, similar to taking an average of classifiers (section 18.4), this time averages over kernels and frees us from the need to choose one particular kernel. It is also possible to take a weighted sum and also learn the weights from data (Lanckriet et al. 2004; Sonnenburg et al. 2006):

$$(14.37) \quad K(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i=1}^{m} \eta_i K_i(\boldsymbol{x}, \boldsymbol{y})$$

subject to $\eta_i \geq 0$, with or without the constraint of $\sum_i \eta_i = 1$, respectively known as convex or conic combination. This is called ***multiple kernel learning*** where we replace a single kernel with a weighted sum (Gönen and Alpaydın 2011). The single kernel objective function of equation 14.27 becomes

$$(14.38) \quad L_d = \sum_t \alpha^t - \frac{1}{2} \sum_t \sum_s \alpha^t \alpha^s r^t r^s \sum_i \eta_i K_i(\boldsymbol{x}^t, \boldsymbol{x}^s)$$

which we solve for both the support vector machine parameters α^t and the kernel weights η_i. Then, the combination of multiple kernels also appear in the discriminant

$$(14.39) \quad g(\boldsymbol{x}) = \sum_t \alpha^t r^t \sum_i \eta_i K_i(\boldsymbol{x}^t, \boldsymbol{x})$$

After training, η_i will take values depending on how the corresponding kernel $K_i(\boldsymbol{x}^t, \boldsymbol{x})$ is useful in discriminating. It is also possible to localize kernels by defining kernel weights as a parameterized function of the input $\boldsymbol{x}$, rather like the gating function in mixture of experts (section 18.8)

$$(14.40) \quad g(\boldsymbol{x}) = \sum_t \alpha^t r^t \sum_i \eta_i(\boldsymbol{x}|\theta_i) K_i(\boldsymbol{x}^t, \boldsymbol{x})$$

and the gating parameters θ_i are learned together with the support vector machine parameters (Gönen and Alpaydın 2008).

When we have information coming from multiple sources in different representations or modalities—for example, in speech recognition where we may have both acoustic and visual lip image—the usual approach is to feed them separately to different classifiers and then fuse the decisions; we will discuss methods for this in detail in chapter 18. Combining multiple kernels provides us with another way of integrating input from multiple sources, where there is a single classifier that uses different kernels for inputs of different sources, for which there are different notions of similarity (Noble 2004). The localized version can then be seen as an extension of this where we can choose between sources, and hence similarity measures, depending on the input.

14.9 Multiclass Kernel Machines

When there are $K > 2$ classes, the straightforward, *one-vs.-all* way is to define K two-class problems, each one separating one class from all other classes combined and learn K support vector machines $g_i(\boldsymbol{x})$, $i = 1, \ldots, K$. That is, in training $g_i(\boldsymbol{x})$, examples of C_i are labeled +1 and examples of C_k, $k \neq i$ are labeled as −1. During testing, we calculate all $g_i(\boldsymbol{x})$ and choose the maximum.

Platt (1999) proposed to fit a sigmoid to the output of a single (2-class) SVM output to convert to a posterior probability. Similarly, one can train one layer of softmax outputs to minimize cross-entropy to generate $K > 2$ posterior probabilities (Mayoraz and Alpaydın 1999):

$$y_i(\boldsymbol{x}) = \sum_{j=1}^{K} v_{ij} f_j(\boldsymbol{x}) + v_{i0} \tag{14.41}$$

where $f_j(\boldsymbol{x})$ are the SVM outputs and y_i are the posterior probability outputs. Weights v_{ij} are trained to minimize cross-entropy. Note, however, that as in stacking (section 18.9), the data on which we train v_{ij} should be different from the data used to train the base SVMs $f_j(\boldsymbol{x})$, to alleviate overfitting.

Instead of the usual approach of building K two-class SVM classifiers to separate one from all the rest, as with any other classifier, one can build $K(K-1)/2$ *pairwise* classifiers (see also section 10.4), each $g_{ij}(\boldsymbol{x})$ taking examples of C_i with the label +1, examples of C_j with the label −1, and not using examples of the other classes. Separating classes in pairs is normally expected to be an easier job, with the additional advantage that because we use less data, the optimizations will be faster, noting however that we have $O(K^2)$ discriminants to train instead of $O(K)$.

In the general case, both one-vs.-all and pairwise separation are special cases of the ***error-correcting output codes*** (ECOC) that decompose a multiclass problem to a set of two-class problems (Dietterich and Bakiri 1995) (see also section 18.6). SVMs being two-class classifiers are ideally suited to this (Allwein, Schapire, and Singer 2000), and it is also possible to have an incremental approach where new two-class SVMs are added to better separate pairs of classes that are confused, to ameliorate a poor ECOC matrix (Mayoraz and Alpaydın 1999).

Another possibility is to write a single *multiclass* optimization problem involving all classes (Weston and Watkins 1998):

$$\min \frac{1}{2}\sum_{i=1}^{K}\|\boldsymbol{w}_i\|^2 + C\sum_i\sum_t \xi_i^t \tag{14.42}$$

subject to

$$\boldsymbol{w}_{z^t}\boldsymbol{x}^t + w_{z^t 0} \geq \boldsymbol{w}_i\boldsymbol{x}^t + w_{i0} + 2 - \xi_i^t, \forall i \neq z^t \text{ and } \xi_i^t \geq 0$$

where z^t contains the class index of $\boldsymbol{x}^t$. The regularization terms minimize the norms of all hyperplanes simultaneously, and the constraints are there to make sure that the margin between the actual class and any other class is at least 2. The output for the correct class should be at least +1, the output of any other class should be at least −1, and the slack variables are defined to make up any difference.

Though this looks neat, the one-vs.-all approach is generally preferred because it solves K separate N variable problems whereas the multiclass formulation uses $K \cdot N$ variables.

14.10 Kernel Machines for Regression

Now let us see how support vector machines can be generalized for regression. We see that the same approach of defining acceptable margins, slacks, and a regularizing function that combines smoothness and error is also applicable here. We start with a linear model, and later on we see how we can use kernel functions here as well:

$$f(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + w_0$$

In regression proper, we use the square of the difference as error:

$$e_2(r^t, f(\boldsymbol{x}^t)) = [r^t - f(\boldsymbol{x}^t)]^2$$

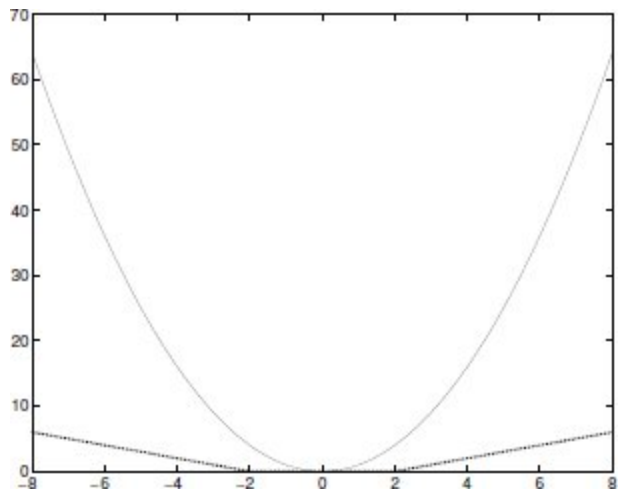


Figure 14.6 Quadratic and ϵ-sensitive error functions. We see that ϵ-sensitive error function is not affected by small errors and also is less affected by large errors and thus is more robust to outliers.

whereas in support vector regression, we use the ϵ-sensitive loss function:

$$e_\epsilon(r^t, f(\mathbf{x}^t)) = \begin{cases} 0 & \text{if } |r^t - f(\mathbf{x}^t)| < \epsilon \\ |r^t - f(\mathbf{x}^t)| - \epsilon & \text{otherwise} \end{cases} \tag{14.43}$$

which means that we tolerate errors up to ϵ and also that errors beyond have a linear effect and not a quadratic one. This error function is therefore more tolerant to noise and is thus more ***robust*** (see figure 14.6). As in the hinge loss, there is a region of no error, which causes sparseness.

Analogous to the soft margin hyperplane, we introduce slack variables to account for deviations out of the ϵ-zone and we get (Vapnik 1995)

$$\min \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_t(\xi_+^t + \xi_-^t) \tag{14.44}$$

subject to

$$
\begin{aligned}
r^t - (\boldsymbol{w}^T\boldsymbol{x} + w_0) &\leq \epsilon + \xi_+^t \\
(\boldsymbol{w}^T\boldsymbol{x} + w_0) - r^t &\leq \epsilon + \xi_-^t \\
\xi_+^t, \xi_-^t &\geq 0
\end{aligned}
$$

where we use two types of slack variables, for positive and negative deviations, to keep them positive. Actually, we can see this as two hinges added back to back, one for positive and one for negative slacks. This formulation corresponds to the ϵ-sensitive loss function given in equation 14.43. The Lagrangian is

$$
\begin{aligned}
L_p &= \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_t(\xi_+^t + \xi_-^t) \\
&\quad - \sum_t \alpha_+^t \left[\epsilon + \xi_+^t - r^t + (\boldsymbol{w}^T\boldsymbol{x} + w_0)\right] \\
&\quad - \sum_t \alpha_-^t \left[\epsilon + \xi_-^t + r^t - (\boldsymbol{w}^T\boldsymbol{x} + w_0)\right] \\
&\quad - \sum_t (\mu_+^t \xi_+^t + \mu_-^t \xi_-^t) \qquad (14.45)
\end{aligned}
$$

Taking the partial derivatives, we get

$$
\frac{\partial L_p}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_t(\alpha_+^t - \alpha_-^t)\boldsymbol{x}^t = 0 \Rightarrow \boldsymbol{w} = \sum_t(\alpha_+^t - \alpha_-^t)\boldsymbol{x}^t \qquad (14.46)
$$

$$
\frac{\partial L_p}{\partial w_0} = \sum_t(\alpha_+^t - \alpha_-^t) = 0 \qquad (14.47)
$$

$$
\frac{\partial L_p}{\partial \xi_+^t} = C - \alpha_+^t - \mu_+^t = 0 \qquad (14.48)
$$

$$
\frac{\partial L_p}{\partial \xi_-^t} = C - \alpha_-^t - \mu_-^t = 0 \qquad (14.49)
$$

The dual is

$$
\begin{aligned}
L_d &= -\frac{1}{2}\sum_t\sum_s(\alpha_+^t - \alpha_-^t)(\alpha_+^s - \alpha_-^s)(\boldsymbol{x}^t)^T\boldsymbol{x}^s \\
&\quad -\epsilon\sum_t(\alpha_+^t + \alpha_-^t) + \sum_t r^t(\alpha_+^t - \alpha_-^t) \qquad (14.50)
\end{aligned}
$$

subject to

$$0 \le \alpha_+^t \le C\,,\ 0 \le \alpha_-^t \le C\,,\ \sum_t (\alpha_+^t - \alpha_-^t) = 0$$

Once we solve this, we see that all instances that fall in the tube have $\alpha_+^t = \alpha_-^t = 0$; these are the instances that are fitted with enough precision (see figure 14.7). The support vectors satisfy either $\alpha_+^t > 0$ or $\alpha_-^t > 0$ and are of two types. They may be instances that are on the boundary of the tube (either α_+^t or α_-^t is between 0 and C), and we use these to calculate w_0. For example, assuming that $\alpha_+^t > 0$, we have $r^t = \boldsymbol{w}^T\boldsymbol{x}^t + w_0 + \epsilon$. Instances that fall outside the ϵ-tube are of the second type; these are instances for which we do not have a good fit $(\alpha_+^t = C)$, as shown in figure 14.7.

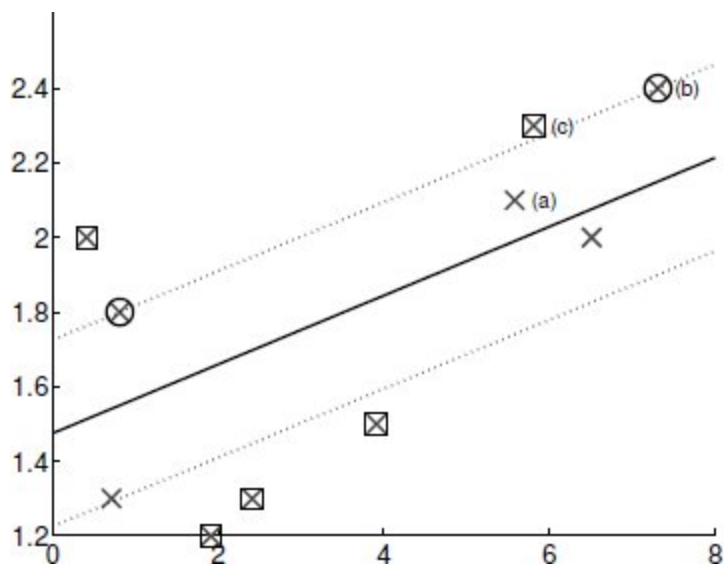


Figure 14.7 The fitted regression line to data points shown as crosses and the ϵ-tube are shown ($C = 10$, $\epsilon = 0.25$). There are three cases: In (a), the instance is in the tube; in (b), the instance is on the boundary of the tube (circled instances); in (c), it is outside the tube with a positive slack, that is, $\xi_+^t > 0$ (squared instances). (b) and (c) are support vectors. In terms of the dual variable, in (a), $\alpha_+^t = 0$, $\alpha_-^t = 0$, in (b), $\alpha_+^t < C$, and in (c), $\alpha_+^t = C$.

Using equation 14.46, we can write the fitted line as a weighted sum of the support vectors:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 = \sum_t (\alpha_+^t - \alpha_-^t)(\mathbf{x}^t)^T\mathbf{x} + w_0 \tag{14.51}$$

Again, the dot product $(\mathbf{x}^t)^T\mathbf{x}^s$ in equation 14.50 can be replaced with a kernel $K(\mathbf{x}^t, \mathbf{x}^s)$, and similarly $(\mathbf{x}^t)^T\mathbf{x}$ be replaced with $K(\mathbf{x}^t, \mathbf{x})$ and we can have a nonlinear fit. Using a polynomial kernel would be similar to fitting a polynomial (figure 14.8), and using a Gaussian kernel (figure 14.9) would be similar to nonparametric smoothing models (section 8.8) except that because of the sparsity of solution, we would not need the whole training set but only a subset.

There is also an equivalent ν-SVM formulation for regression (Schölkopf et al. 2000), where instead of fixing ϵ, we fix ν to bound the fraction of support vectors. There is still a need for C though.

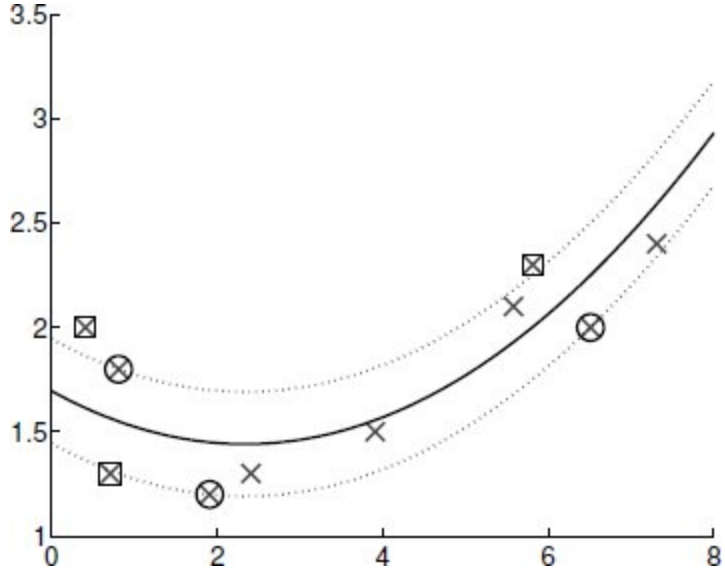


Figure 14.8 The fitted regression line and the ϵ-tube using a quadratic kernel are shown (C = 10, ϵ = 0.25). Circled instances are the support vectors on the margins, squared instances are support vectors which are outliers.

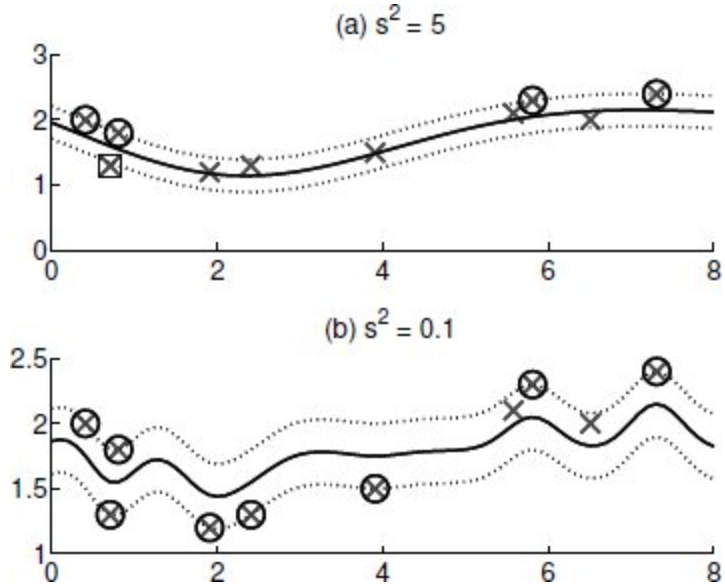


Figure 14.9 The fitted regression line and the ϵ-tube using a Gaussian kernel with two different spreads are shown (C = 10, ϵ = 0.25). Circled instances are the support vectors on the margins, and squared instances are support vectors that are outliers.

14.11 Kernel Machines for Ranking

Remember that in ranking, we have instances that need to be ordered in a certain way (Liu 2011). For example, we may have pairwise constraints such as $r^u \prec r^v$ which means that instance $\boldsymbol{x}^u$ should generate a higher score than $\boldsymbol{x}^v$. In section 10.8, we discuss how we can train a linear model for this purpose using gradient descent. We now discuss how we can do the same using support vector machines.

We consider each pairwise constraint as one data instance t : $r^u \prec r^v$ and minimize

$$L_p = \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_t \xi^t \tag{14.52}$$

subject to

$$(14.53) \quad \begin{aligned} \boldsymbol{w}^T\boldsymbol{x}^u &\geq \boldsymbol{w}^T\boldsymbol{x}^v + 1 - \xi^t, \text{ for each } t : r^u \prec r^v \\ \xi^t &\geq 0 \end{aligned}$$

Equation 14.53 requires that the score for $\boldsymbol{x}^u$ be at least 1 unit more than the score for $\boldsymbol{x}^v$ and hence defines a margin. If the constraint is not satisfied, the slack variable is nonzero and equation 14.52 minimizes the sum of such slacks and the complexity term, which again corresponds to making the width of the margin as large as possible (Herbrich, Obermayer, and Graepel 2000; Joachims 2002). Note that the second term of the sum of slacks is the same as the error used in equation 10.42 except for the 1 unit margin, and the complexity term, as we discussed before, can be interpreted as a weight decay term on the linear model (see section 12.4.2).

Note that there is one constraint for each pair where an ordering is defined, and hence the number of such constraints is $O(N^2)$. The constraint of equation 14.53 can also be written as

$$\boldsymbol{w}^T(\boldsymbol{x}^u - \boldsymbol{x}^v) \geq 1 - \xi^t$$

That is, we can view this as a two-class classification of pairwise differences, $\boldsymbol{x}^u - \boldsymbol{x}^v$. So by calculating such differences and labeling them as $r^t \in \{-1, +1\}$ depending on whether $r^v \prec r^u$ or $r^u \prec r^v$ respectively, any two-class kernel machine can be used to implement ranking. But this is not the most efficient way to implement, and faster methods have been proposed (Chapelle and Keerthi 2010).

The dual is

$$(14.54) \quad L_d = \sum_t \alpha^t - \frac{1}{2}\sum_t\sum_s \alpha^t\alpha^s(\boldsymbol{x}^u - \boldsymbol{x}^v)^T(\boldsymbol{x}^k - \boldsymbol{x}^l)$$

subject to $0 \leq \alpha^t \leq C$. Here, t and s are two pairwise constraints, such as $t : r^u \prec r^v$ and $s : r^k \prec r^l$. Solving this, for the constraints that are satisfied, we have $\xi^t = 0$ and $\alpha^t = 0$; for the ones that are satisfied but are in the margin, we have $0 < \xi^t < 1$ and $\alpha^t < C$; and for the ones that are not satisfied (and are misranked), we have $\xi^t > 1$ and $\alpha^t = C$.

For new test instance $\boldsymbol{x}$, the score is calculated as

$$g(\boldsymbol{x}) = \sum_t \alpha^t (\boldsymbol{x}^u - \boldsymbol{x}^v)^T \boldsymbol{x} \tag{14.55}$$

It is straightforward to write the kernelized version of the primal, dual, and score functions, and this is left to the reader (see exercise 7).

14.12 One-Class Kernel Machines

Support vector machines, originally proposed for classification, are extended to regression by defining slack variables for deviations around the regression line, instead of the discriminant. We now see how SVM can be used for a restricted type of unsupervised learning, namely, for estimating regions of high density. We are not doing a full density estimation; rather, we want to find a boundary (so that it reads like a classification problem) that separates volumes of high density from volumes of low density (Tax and Duin 1999). Such a boundary can then be used for *novelty* or ***outlier detection***. This is also called ***one-class classification***.

We consider a sphere with center $\boldsymbol{a}$ and radius R that we want to enclose as much as possible of the density, measured empirically as the enclosed training set percentage. At the same time, trading off with it, we want to find the smallest radius (see figure 14.10). We define slack variables for instances that lie outside (we only have one type of slack

variable because we have examples from one class and we do not have any penalty for those inside), and we have a smoothness measure that is proportional to the radius:

$$\min R^2 + C \sum_t \xi^t \tag{14.56}$$

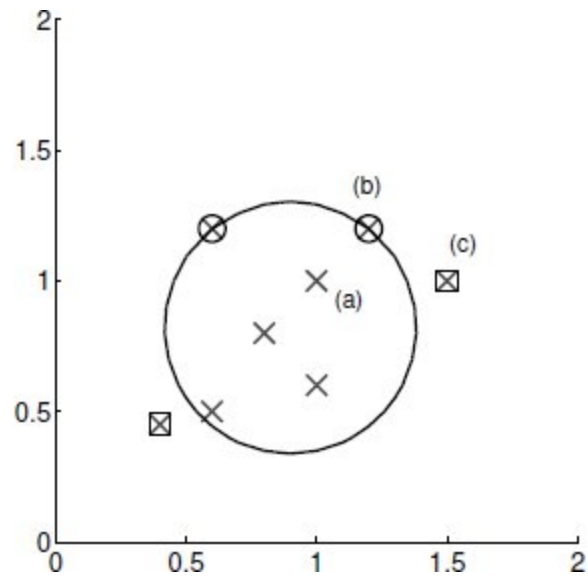


Figure 14.10 One-class support vector machine places the smoothest boundary (here using a linear kernel, the circle with the smallest radius) that encloses as much of the instances as possible. There are three possible cases: In (a), the instance is a typical instance. In (b), the instance falls on the boundary with $\xi^t = 0$; such instances define R. In (c), the instance is an outlier with $\xi^t > 0$. (b) and (c) are support vectors. In terms of the dual variable, we have, in (a), $\alpha^t = 0$; in (b), $0 < \alpha^t < C$; in (c), $\alpha^t = C$.

subject to

$$\|\boldsymbol{x}^t - \boldsymbol{a}\|^2 \leq R^2 + \xi^t \text{ and } \xi^t \geq 0, \forall t$$

Adding the constraints, we get the Lagrangian, which we write keeping in mind that $\|\boldsymbol{x}^t - \boldsymbol{a}\|^2 = (\boldsymbol{x}^t - \boldsymbol{a})^T(\boldsymbol{x}^t - \boldsymbol{a})$:

$$L_p = R^2 + C\sum_t \xi^t - \sum_t \alpha^t \left(R^2 + \xi^t - \left[(\boldsymbol{x}^t)^T\boldsymbol{x}^t - 2\boldsymbol{a}^T\boldsymbol{x}^t + \boldsymbol{a}^T\boldsymbol{a}\right]\right) - \sum_t \gamma^t\xi^t \tag{14.57}$$

with $\alpha^t \geq 0$ and $\gamma^t \geq 0$ being the Lagrange multipliers. Taking the derivative with respect to the parameters, we get

$$\frac{\partial L}{\partial R} = 2R - 2R\sum_t \alpha^t = 0 \Rightarrow \sum_t \alpha^t = 1 \tag{14.58}$$

$$\frac{\partial L}{\partial \boldsymbol{a}} = \sum_t \alpha^t(2\boldsymbol{x}^t - 2\boldsymbol{a}) = 0 \Rightarrow \boldsymbol{a} = \sum_t \alpha^t\boldsymbol{x}^t \tag{14.59}$$

$$\frac{\partial L}{\partial \xi^t} = C - \alpha^t - \gamma^t = 0 \tag{14.60}$$

Since $\gamma^t \geq 0$, we can write this last as the constraint: $0 \leq \alpha^t \leq C$. Plugging these into equation 14.57, we get the dual that we maximize with respect to α^t:

$$L_d = \sum_t \alpha^t(\boldsymbol{x}^t)^T\boldsymbol{x}^t - \sum_t\sum_s \alpha^t\alpha^s(\boldsymbol{x}^t)^T\boldsymbol{x}^s \tag{14.61}$$

subject to

$$0 \leq \alpha^t \leq C \text{ and } \sum_t \alpha^t = 1$$

When we solve this, we again see that most of the instances vanish with their $\alpha^t = 0$; these are the typical, highly likely instances that fall inside the sphere (figure 14.10). There are two types of support vectors with $\alpha^t > 0$: There are instances that satisfy $0 < \alpha^t < C$ and lie on the boundary, $\|\boldsymbol{x}^t - \boldsymbol{a}\|^2 = R^2$ ($\xi^t = 0$), which we use to calculate R. Instances that satisfy $\alpha^t = C$ ($\xi^t > 0$) lie outside the boundary and are the outliers. From equation 14.59, we see that the center $\boldsymbol{a}$ is written as a weighted sum of the support vectors.

Then given a test input $\boldsymbol{x}$, we say that it is an outlier if

$$\|\boldsymbol{x} - \boldsymbol{a}\|^2 > R^2$$

or

$$\boldsymbol{x}^t\boldsymbol{x} - 2\boldsymbol{a}^T\boldsymbol{x} + \boldsymbol{a}^T\boldsymbol{a} > R^2$$

Using kernel functions, allow us to go beyond a sphere and define boundaries of arbitrary shapes. Replacing the dot product with a kernel function, we get (subject to the same constraints):

$$L_d = \sum_t \alpha^t K(\boldsymbol{x}^t, \boldsymbol{x}^t) - \sum_t \sum_s \alpha^t \alpha^s K(\boldsymbol{x}^t, \boldsymbol{x}^s) \tag{14.62}$$

For example, using a polynomial kernel of degree 2 allows arbitrary quadratic surfaces to be used. If we use a Gaussian kernel (equation 14.30), we have a union of local spheres. We reject $\boldsymbol{x}$ as an outlier if

$$K(\boldsymbol{x}, \boldsymbol{x}) - 2\sum_t \alpha^t K(\boldsymbol{x}, \boldsymbol{x}^t) + \sum_t \sum_s \alpha^t \alpha^s K(\boldsymbol{x}^t, \boldsymbol{x}^s) > R^2$$

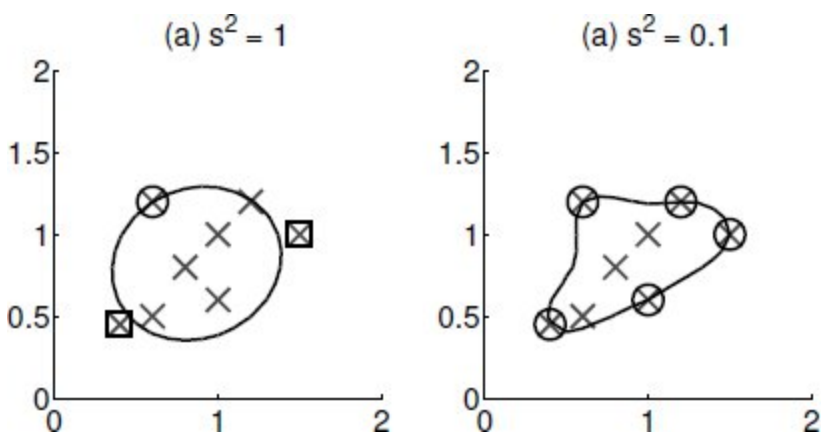


Figure 14.11 One-class support vector machine using a Gaussian kernel with different spreads.

The third term does not depend on $\boldsymbol{x}$ and is therefore a constant (we use this as an equality to solve for R where $\boldsymbol{x}$ is

an instance on the margin). In the case of a Gaussian kernel where $K(\boldsymbol{x}, \boldsymbol{x}) = 1$, the condition reduces to

$$\sum_t \alpha^t K_G(\boldsymbol{x}, \boldsymbol{x}^t) < R_c$$

for some constant R_C, which is analogous to the kernel density estimator (section 8.2.2)—except for the sparseness of the solution—with a probability threshold R_C (see figure 14.11).

There is also an alternative, equivalent ν-SVM type of formulation of one-class support vector machines that uses the canonical $(1/2)\|\boldsymbol{w}\|^2$ type of smoothness (Schölkopf et al. 2001).

14.13 Large Margin Nearest Neighbor Classifier

In chapter 8, we discussed nonparametric methods where instead of fitting a global model to the data we interpolate from a subset of neighboring instances, and specifically in section 8.6, we covered the importance of using a good distance measure. We now discuss a method to learn a distance measure from the data. Strictly speaking, this is not a kernel machine, but it uses the idea of keeping a margin in ranking, as we noted in section 14.11.

The basic idea is to view k-nearest neighbor classification (section 8.4) as a ranking problem. Let us say the k-nearest neighbors of $\boldsymbol{x}^i$ contains two instances $\boldsymbol{x}^j$ and $\boldsymbol{x}^l$ such that $\boldsymbol{x}^i$ and $\boldsymbol{x}^j$ are of the same class and $\boldsymbol{x}^l$ belongs to another class. In such a case, we want a distance measure such that the distance between $\boldsymbol{x}^i$ and $\boldsymbol{x}^l$ is more than $\boldsymbol{x}^i$ and $\boldsymbol{x}^j$. Actually, we not only require that it be more but that there be a one-unit margin between them and if this is not satisfied, we have a slack variable for the difference:

$$\mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^l) \geq \mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^j) + 1 - \xi^{ijl}$$

The distance measure works as a score function in a ranking problem, and each $(\boldsymbol{x}^i, \boldsymbol{x}^j, \boldsymbol{x}^l)$ triple defines one ranking constraint as in equation 14.53.

This is the basic idea behind the ***large margin nearest neighbor*** (LMNN) algorithm (Weinberger and Saul 2009). The error function minimized is

$$(14.63) \qquad (1-\mu)\sum_{i,j}\mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^j) + \mu\sum_{i,j,l}(1-y_{il})\xi_{ijl}$$

subject to

$$(14.64) \qquad \begin{aligned} \mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^l) &\geq \mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^j) + 1 - \xi^{ijl}, \text{ if } \boldsymbol{r}^i = \boldsymbol{r}^j \text{ and } \boldsymbol{r}^i \neq \boldsymbol{r}^l \\ \xi^{ijl} &\geq 0 \end{aligned}$$

Here, $\boldsymbol{x}^j$ is one of the k-nearest neighbors of $\boldsymbol{x}^i$ and they are of the same class: $\boldsymbol{r}^i = \boldsymbol{r}^j$—it is a *target* neighbor. $\boldsymbol{x}^l$ is also one of the k-nearest neighbors of $\boldsymbol{x}^i$; if they are of the same label, then y_{il} is set to 1 and we incur no loss; if they are of different classes, then $\boldsymbol{x}^l$ is an *impostor*, y_{il} is set to 0, and if the condition 14.64 is not satisfied, the slack defines a cost. The second term of equation 14.63 is the sum of such slacks. The first term is the total distance to all target neighbors and minimizing that has an effect of regularization—we want to keep the distances as small as possible.

In LMNN, Mahalanobis distance is used as the distance measure model:

$$(14.65) \qquad \mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^j|\mathbf{M}) = (\boldsymbol{x}^i - \boldsymbol{x}^j)^T\mathbf{M}(\boldsymbol{x}^i - \boldsymbol{x}^j)$$

and $\mathbf{M}$ matrix is the parameter that is to be optimized. Equation 14.63 defines a convex (more specifically, positive semi-definite) problem and hence has a unique minimum.

When the input dimensionality is high and there are few data, as we discuss in equation 8.21, we can regularize by factoring $\mathbf{M}$ as $\mathbf{L}^T\mathbf{L}$ where $\mathbf{L}$ is $k \times d$ with $k < d$:

$$\mathcal{D}(\boldsymbol{x}^i, \boldsymbol{x}^j | \mathbf{L}) = \|\mathbf{L}\boldsymbol{x}^i - \mathbf{L}\boldsymbol{x}^j\|^2 \tag{14.66}$$

$\mathbf{L}\boldsymbol{x}$ is the k-dimensional projection of $\boldsymbol{x}$, and Mahalanobis distance in the original d-dimensional $\boldsymbol{x}$ space corresponds to the (squared) Euclidean distance in the new k-dimensional space—see figure 8.7 for an example. If we plug equation 14.66 into equation 14.63 as the distance measure, we get the ***large margin component analysis*** (LMCA) algorithm (Torresani and Lee 2007); unfortunately, this is no longer a convex optimization problem, and if we use gradient descent, we get a locally optimal solution.

14.14 Kernel Dimensionality Reduction

We know from section 6.3 that principal components analysis (PCA) reduces dimensionality by projecting on the eigenvectors of the covariance matrix $\boldsymbol{\Sigma}$ with the largest eigenvalues, which, if data instances are centered ($E[\boldsymbol{x}] = 0$), can be written as $\mathbf{X}^T\mathbf{X}$. In the kernelized version, we work in the space of $\boldsymbol{\phi}(\boldsymbol{x})$ instead of the original $\boldsymbol{x}$ and because, as usual, the dimensionality d of this new space may be much larger than the dataset size N, we prefer to work with the $N \times N$ matrix $\mathbf{X}\mathbf{X}^T$ and do feature embedding instead of working with the $d \times d$ matrix $\mathbf{X}^T\mathbf{X}$. The projected data matrix is $\boldsymbol{\Phi} = \boldsymbol{\phi}(\mathbf{X})$, and hence we work with the eigenvectors of $\boldsymbol{\Phi}^T\boldsymbol{\Phi}$ and hence the kernel matrix $\mathbf{K}$.

Kernel PCA uses the eigenvectors and eigenvalues of the kernel matrix and this corresponds to doing a linear dimensionality reduction in the $\phi(\boldsymbol{x})$ space. When $\boldsymbol{c}_i$ and λ_i are

the corresponding eigenvectors and eigenvalues, the projected new k-dimensional values can be calculated as

$$\mathbf{z}_j^t = \sqrt{\lambda_j}\mathbf{c}_j^t, j = 1, \ldots, k, \; t = 1, \ldots, N$$

An example is given in figure 14.12 where we first use a quadratic kernel and then decrease dimensionality to two (out of five) using kernel PCA and implement a linear SVM there. Note that in the general case (e.g., with a Gaussian kernel), the eigenvalues do not necessarily decay and there is no guarantee that we can reduce dimensionality using kernel PCA.

What we are doing here is multidimensional scaling (section 6.7) using kernel values as the similarity values. For example, by taking $k = 2$, one can visualize the data in the space induced by the kernel matrix, which can give us information as to how similarity is defined by the used kernel. Linear discriminant analysis (LDA) (section 6.8) can similarly be kernelized (Müller et al. 2001). The kernelized version of canonical correlation analysis (CCA) (section 6.9) is discussed in Hardoon, Szedmak, and Shawe-Taylor 2004.

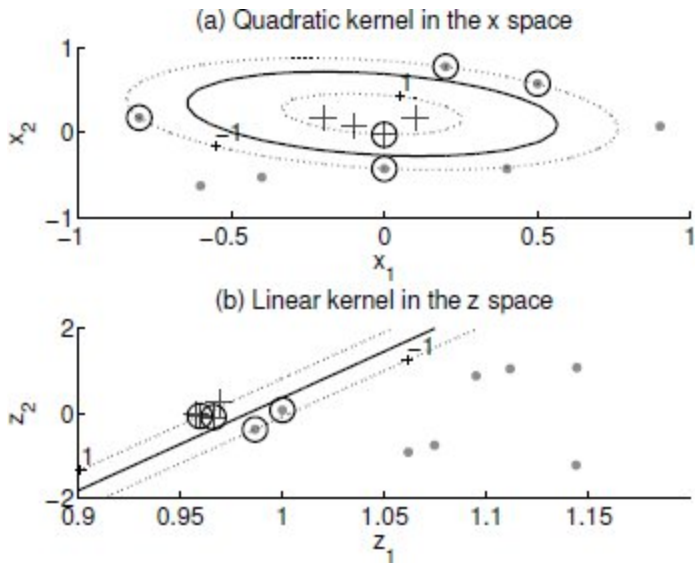


Figure 14.12 Instead of using a quadratic kernel in the original space (a), we can use kernel PCA on the quadratic kernel values to map to a two-dimensional new space where we use a linear discriminant (b); these two dimensions (out of five) explain 80 percent of the variance.

In chapter 6, we discussed nonlinear dimensionality reduction methods, Isomap and LLE. In fact, by viewing the elements of the cost matrix in equation 6.58 as kernel evaluations for pairs of inputs, LLE can be seen as kernel PCA for a particular choice of kernel. The same also holds for Isomap when a kernel function is defined as a function of the geodesic distance on the graph.

14.15 Notes

The idea of generalizing linear models by mapping the data to a new space through nonlinear basis functions is old, but the novelty of support vector machines is that of integrating this into a learning algorithm whose parameters are defined in terms of a subset of data instances (the so-called ***dual representation***), hence also without needing to explicitly